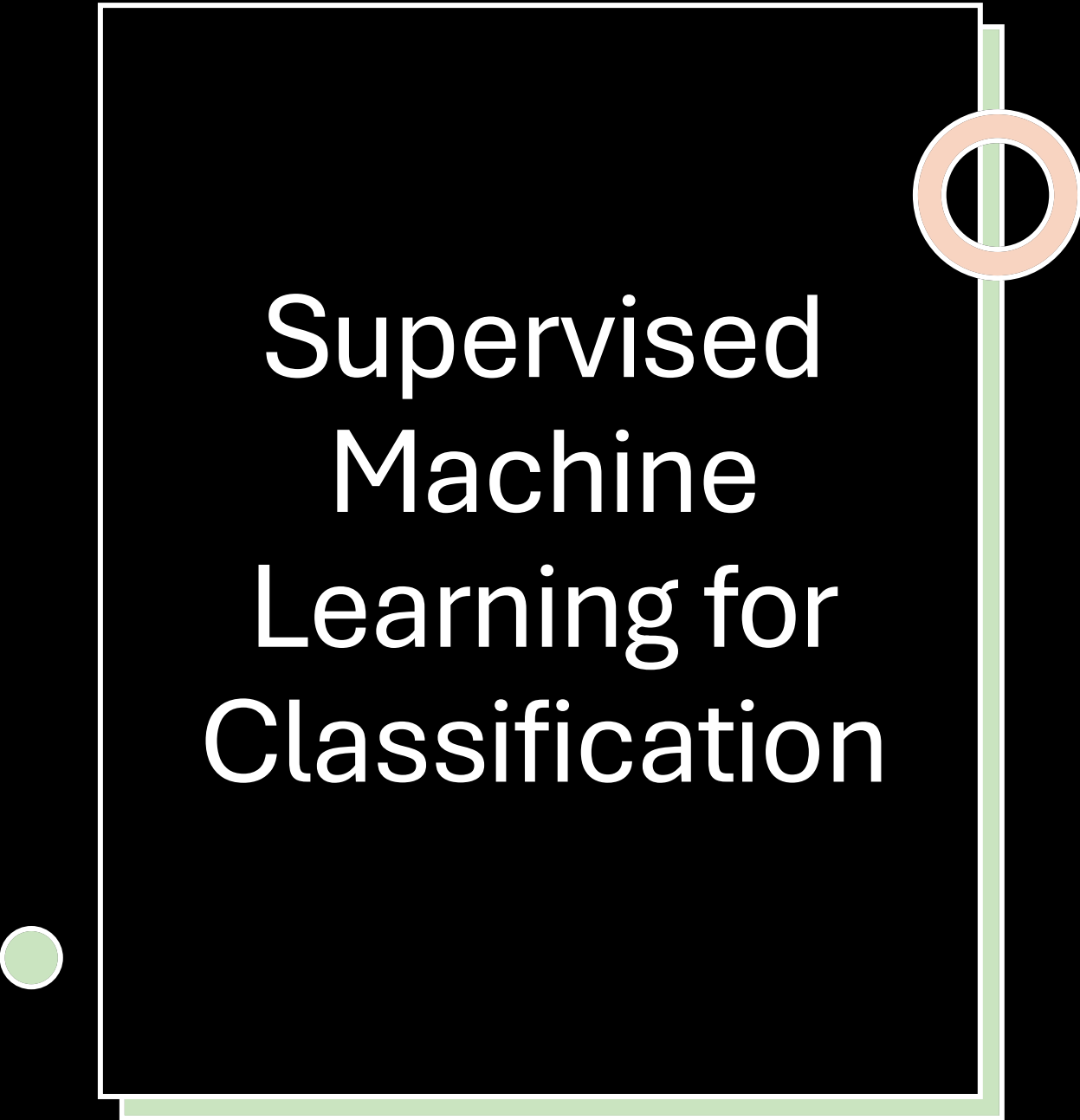
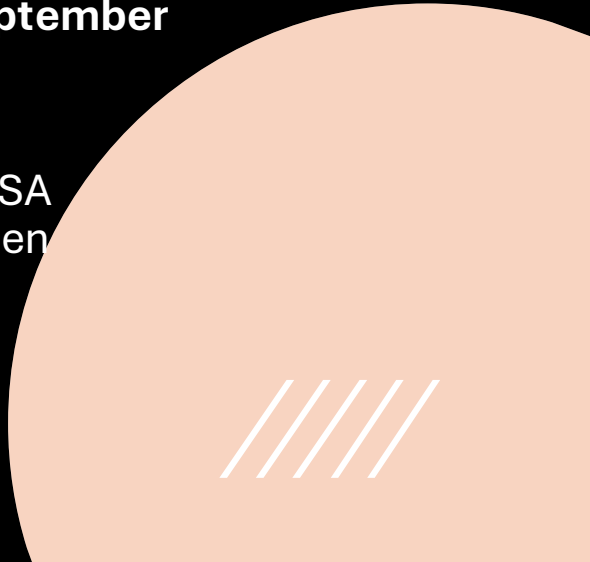




Supervised Machine Learning for Classification

**Viewing Space Through machine
Learning and Astrostatistics
(VISTULA) – 21-25 September
2026, Warsaw**

Antonio La Marca – ESA
Research Fellow, Leiden
University



Outline

First lecture block

- Introduction to supervised classification problems in astronomy
- K-Nearest Neighbours
- Decision Trees
- Ensemble methods
 - Bagging & Random Forest
 - Boosting (e.g. XGBoost)
- Wrap-up

Outline

Second lecture block

- Recap & introduction to Deep Learning
- Neural Networks
- Convolutional Neural Networks (CNNs)
 - CNNs in astronomy
- Wrap-up

References

- **“Statistics, Data Mining, and Machine Learning in Astronomy: A Practical Python Guide for the Analysis of Survey Data, Updated Edition”** – Željko Ivezić, Andrew J. Connolly, Jacob T. VanderPlas, and Alexander Gray
- **“An Introduction to Statistical Learning – with applications in Python”**– Gareth James, Daniela Witten, Trevor Hastie, Robert Tibshirani, and Jonathan Taylor



Bio

- BSc Naples
- MSc Naples – Groningen
 - Working on dwarf galaxies
- PhD Groningen (Prof. Lingyu Wang & Prof. Scott Trager)
 - The connection between galaxy morphology and AGN triggering
 - Machine Learning for image classification
- ESA Postdoctoral Research Fellow at ESTEC (Leiden)

Glossary

Input dataset **X**, made of p variables (**features**):

$$\mathbf{X} = (X_1, X_2, \dots, X_p)$$

If the dataset is made of N observations, X is a N×p matrix (x_{ij})

Output Y (response or dependent variable): quantitative or qualitative (categorical/**label**) variable

Is there a relationship between **Y** and **X**?

$$\mathbf{Y} = f(\mathbf{X}) + \epsilon$$

f is our model (or **classifier**)

SUPERVISED

Known target Y

INPUT Labelled examples pair inputs X with targets Y

OBJECTIVE Learn $f(X) \rightarrow y$ and predict outcomes for unseen cases

TASKS Classification · regression

METHODS k-NN · decision trees / forests · SVM · neural networks

EVALUATION Compare predictions with held-out labels

ASTRONOMY Classify galaxy morphology

UNSUPERVISED

No target label

INPUT Features X without target labels

OBJECTIVE Reveal structure, groups, or compact representations

TASKS Clustering · dimensionality reduction · anomaly detection

METHODS k-means / DBSCAN · hierarchical clustering · PCA · autoencoders

EVALUATION Check stability, compactness, and domain usefulness

ASTRONOMY Find galaxy populations or rare outliers

Labels set the question: **predict** a known outcome, or **uncover** hidden structure.

Supervised Classification

The motivation comes from the long history of classification in astronomy

- Star spectral types
- Star/Galaxy/Quasar separation
- Hubble morphological classification of galaxies

Takes a set of features and relates them to predefined classes

Goal: Characterise the relation between the features and their classes and apply these classifications to unlabelled data

Note: Classes might arise from numerical values.

- E.g. X-ray luminosity threshold to divide AGN from other X-ray sources

Classification Loss

- The most common is the **zero-one loss**

$$L(y, \hat{y}) = \delta(y \neq \hat{y}) = \begin{cases} 1 & \text{if } y \neq \hat{y} \\ 0 & \text{otherwise} \end{cases}$$

The expectation value of this loss is the **misclassification or error rate**

$$\frac{1}{n} \sum_{i=1}^n \delta(y_i \neq \hat{y}_i)$$

In general, we want to select a model that minimises the error rate

Classification – model evaluation

Binary classification setting: class labels 1 and 0

Two types of error

- *False positive*: assigning a label 1 to an object whose true class is 0
- *False negative*: assigning a label 0 to an object whose true class is 1

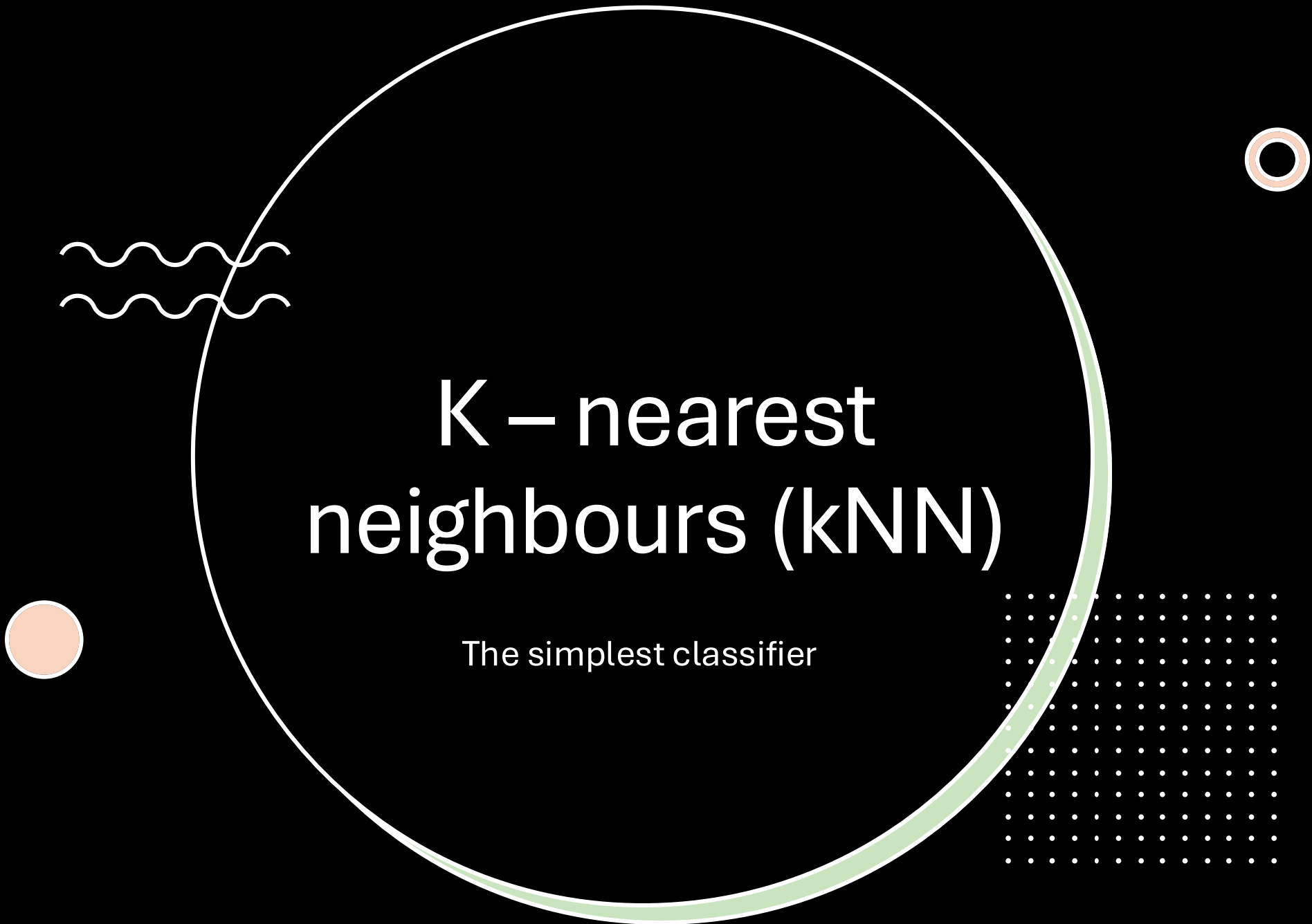
$$\textbf{Completeness} = \frac{\text{true positives}}{\text{true positives} + \text{false negatives}}$$

$$\textbf{Precision} = 1 - \text{contamination} = \frac{\text{true positives}}{\text{true positives} + \text{false positives}}$$

Confusion matrix

		True Class	
		Positive	Negative
Predicated Class	Positive	TP	FP
	Negative	FN	TN

$$\textit{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$



K – nearest neighbours (kNN)

The simplest classifier

KNN – Intuition

Similar objects fall close to each other in the parameter space

Mathematically, the intuitive justification is that $p(y|x) \approx p(y|x')$ if x' is very close to x

The simplicity of KNN is that it assumes nothing about the form of the relation between y and x , it is completely **non-parametric**

For a new observation, just use the class label of the nearest points

KNN – Definition

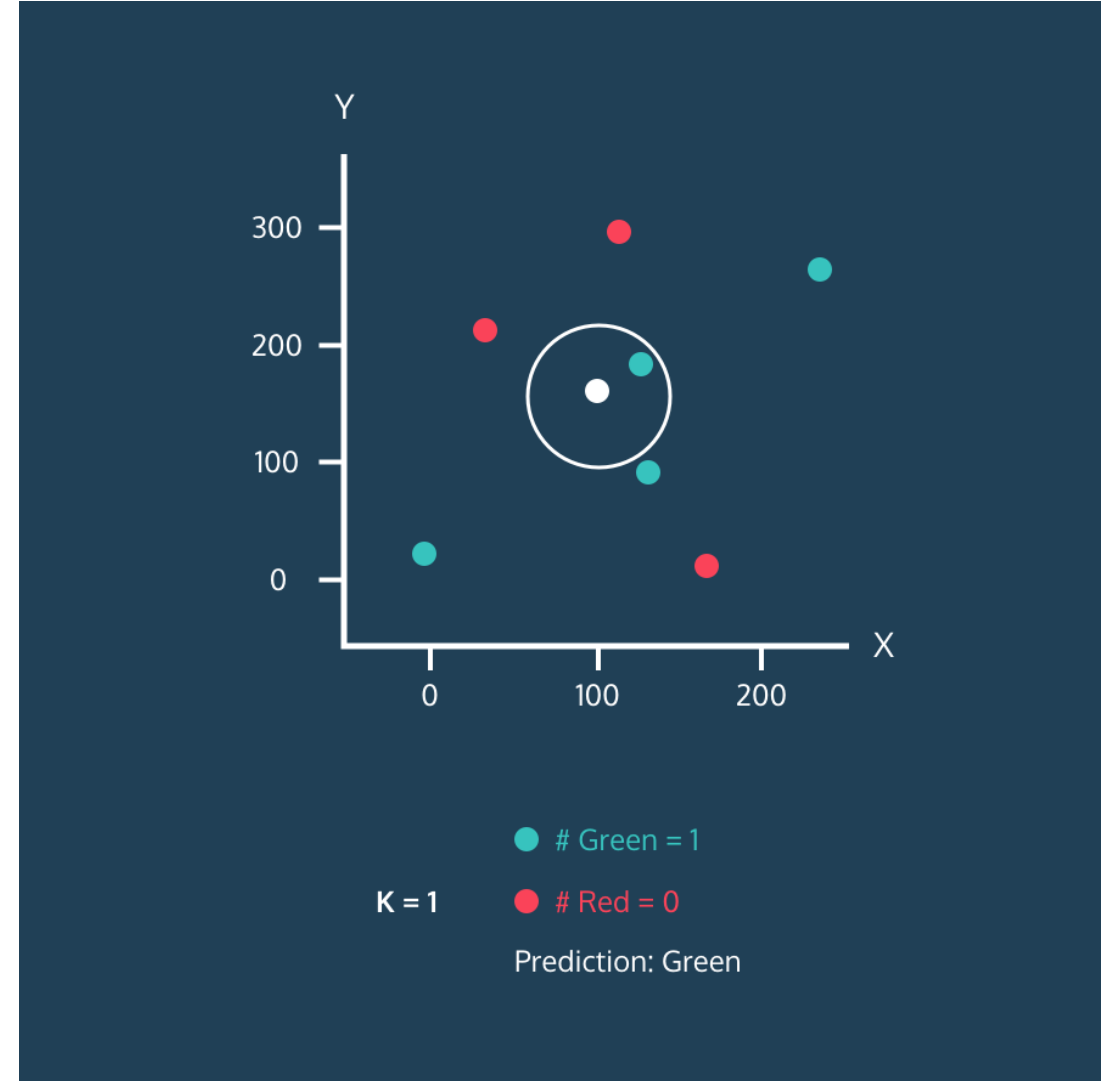
Given an integer $k > 0$, a training dataset, and a test observation x_0 , the kNN classifier identifies the k points in the training dataset that are the closest to x_0 , named N_0 , and then it estimates the conditional probability for class j as the fraction of points in N_0 whose response values equal j :

$$P(y = j|x = x_0) = \frac{1}{K} \sum_{i \in N_0} \delta(y_i = j)$$

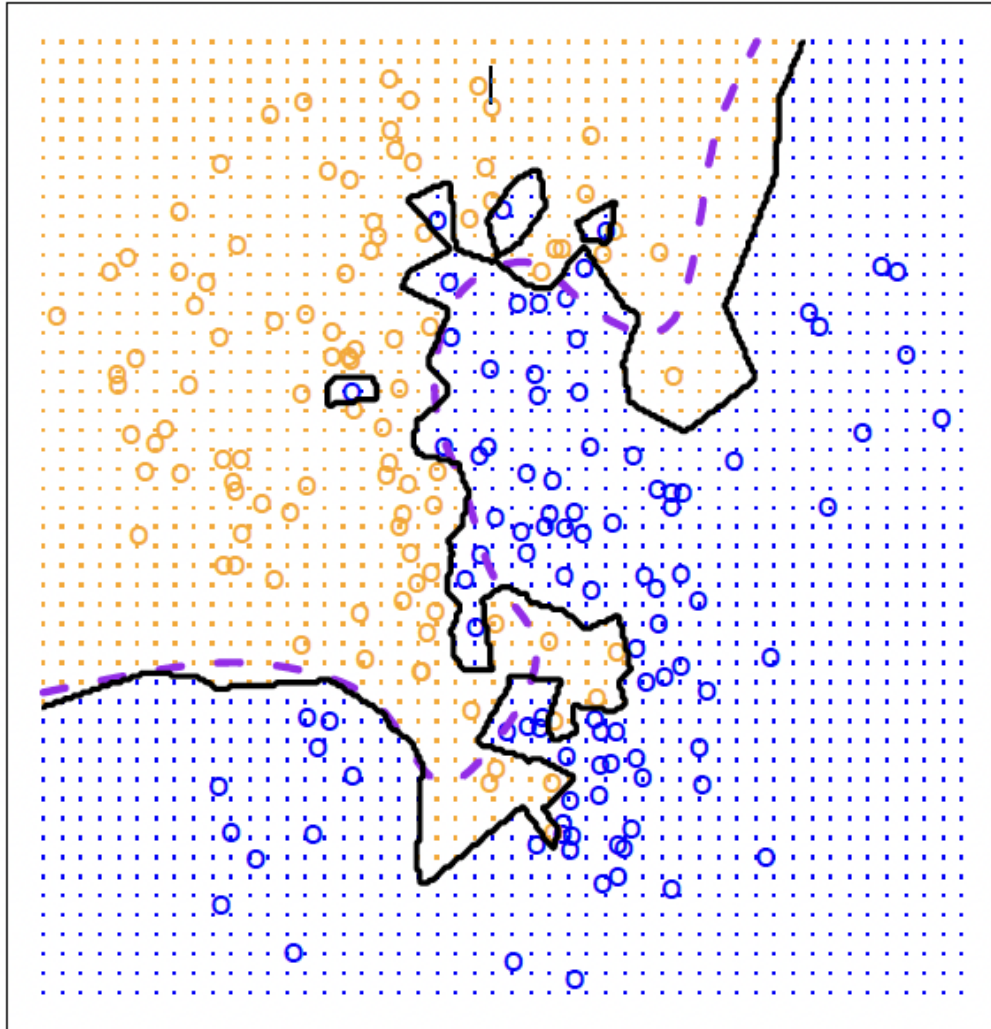
Finally, the kNN classifier classifies the test observation to the class with the largest probability

The choice of k is crucial

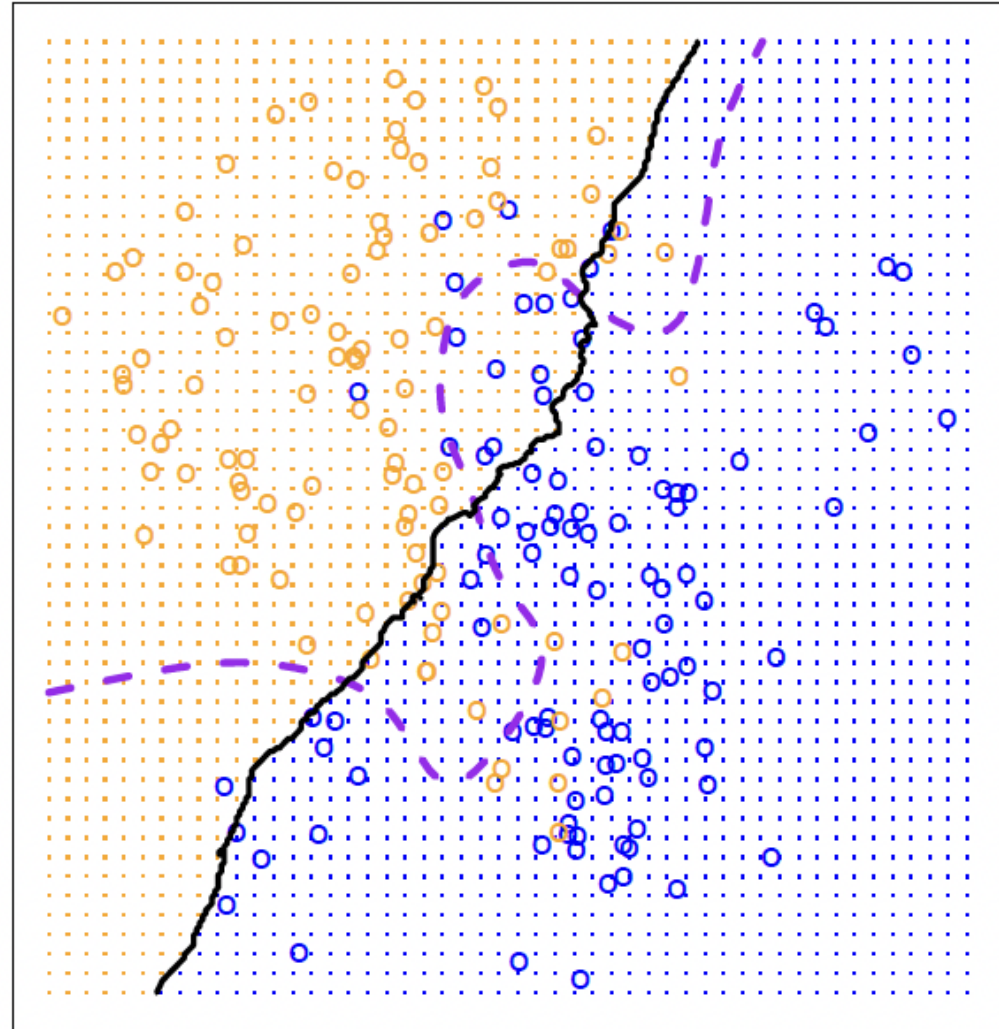
- Is the only hyper-parameter to finetune/train
- Small k it is too flexible and overfits the noise
- As k grows the model becomes less flexible and approximates a linear model
- Cross-validation used to estimate the best k



KNN: $K=1$

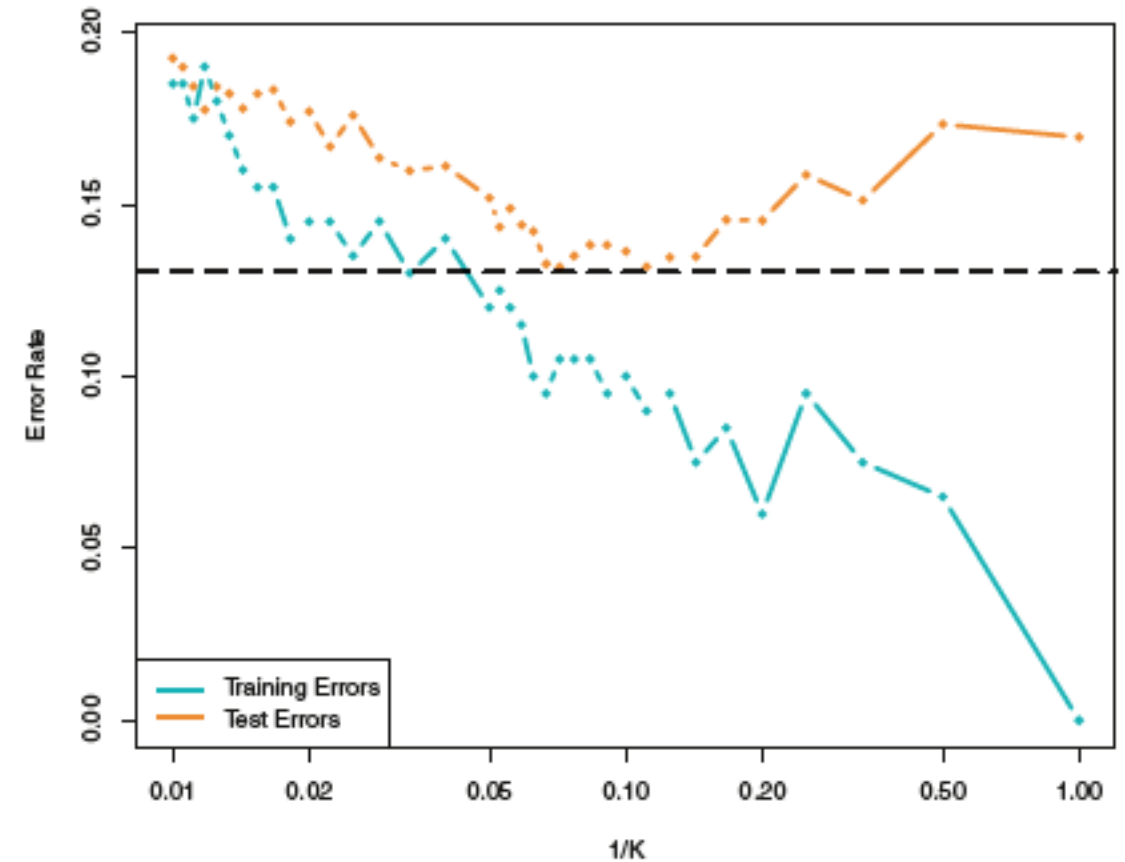


KNN: $K=100$

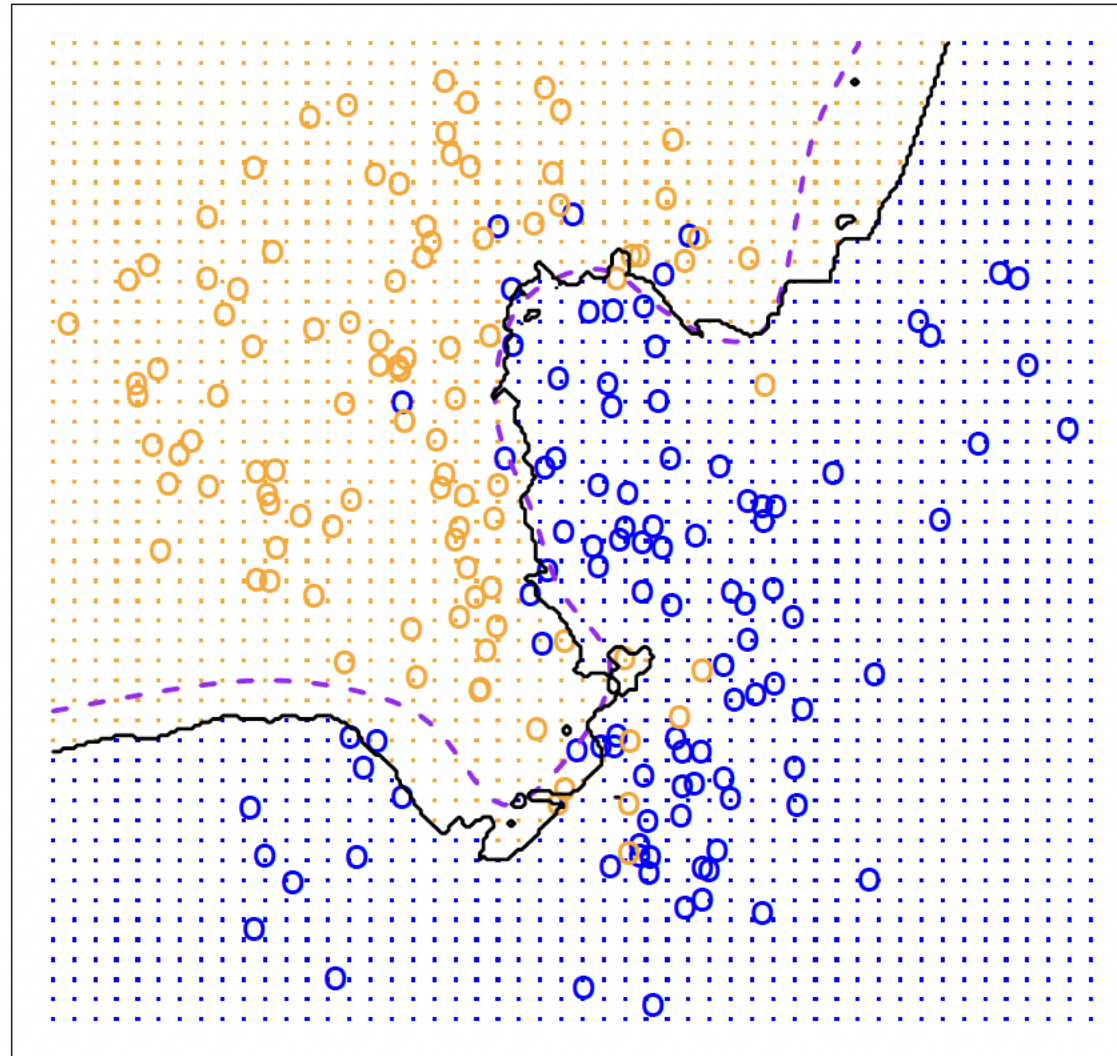


Estimate k with cross-validation

- Split the training data in multiple subsets and in turn each subset act as test set
- Compute the average train and test error rate as a function of k or $1/k$
- “Classic” U-shape for test error: as $1/k$ increases the error declines, but the error increases again after a minimum as the model overfits

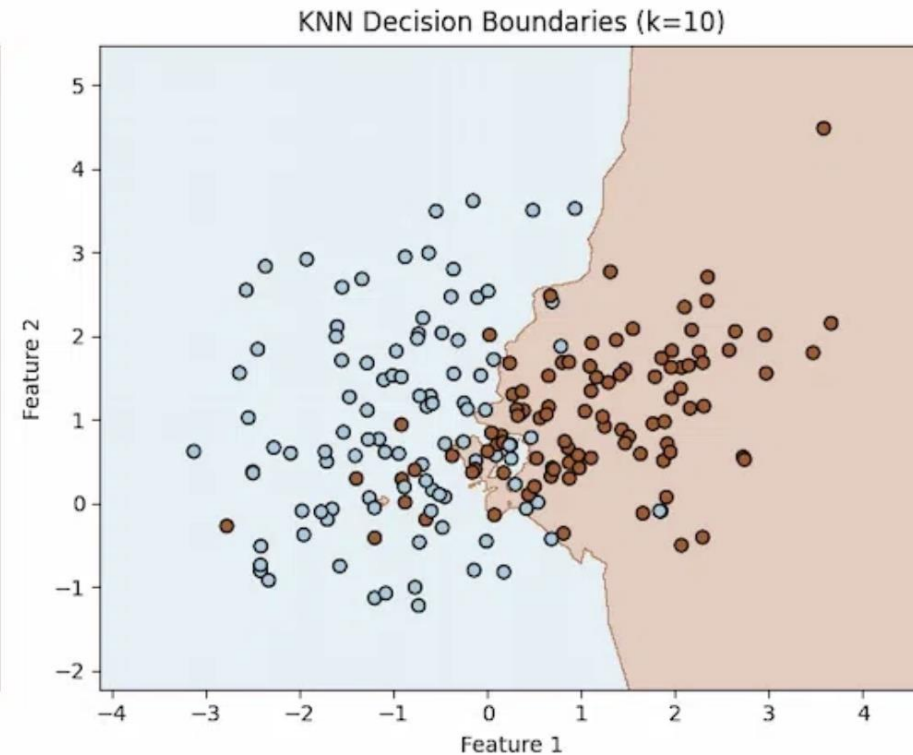
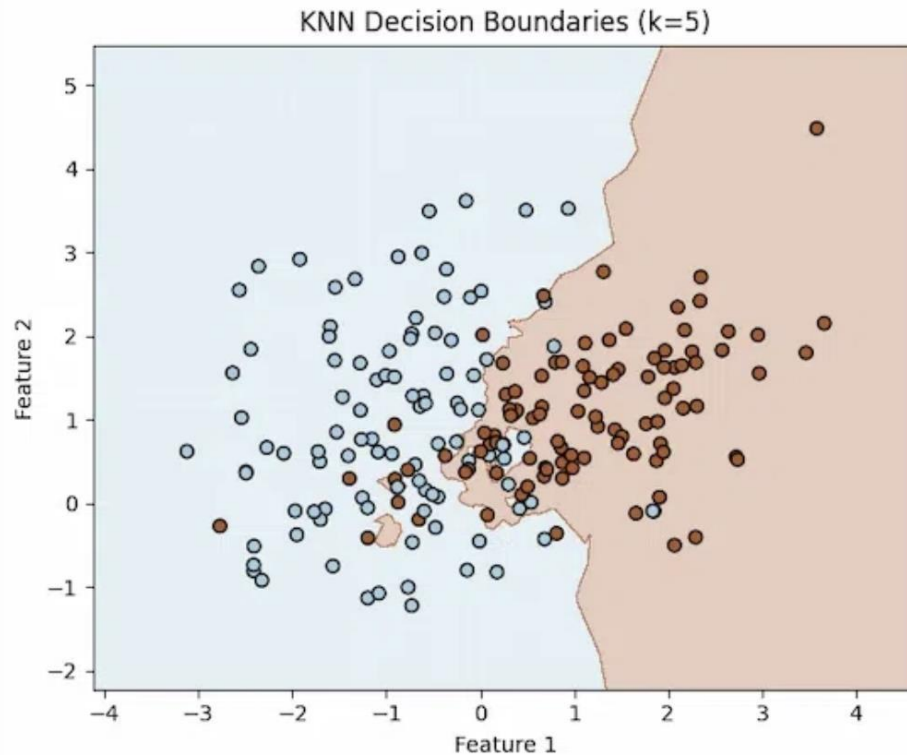


KNN: K=10



Decision boundary

- The resulting decision boundaries partition the space into regions
- Boundaries are generally non-linear
- Classification regions → Voronoi tessellation



Voting rules



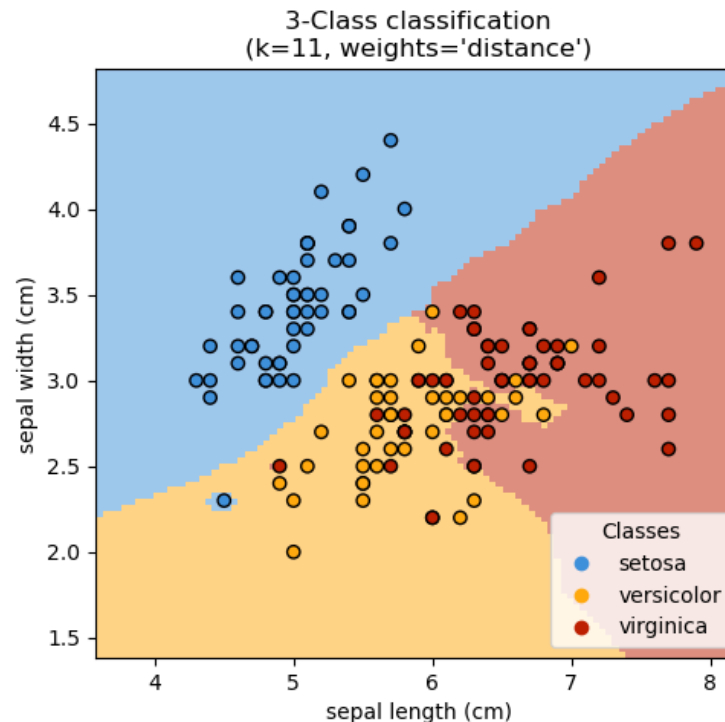
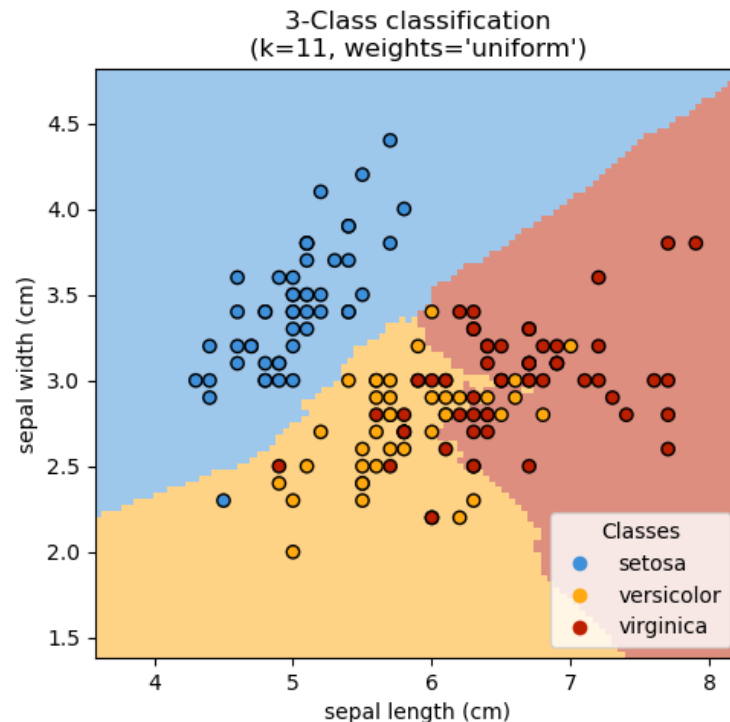
Simplest approach: majority vote rule

E.g. $k=10$, two classes. Three red points, seven blue points $\rightarrow$ blue class



Advanced form: Weighted vote rule

Weight each vote by the distance to the nearest point. The weights are $1/\text{distance}$.



Metrics

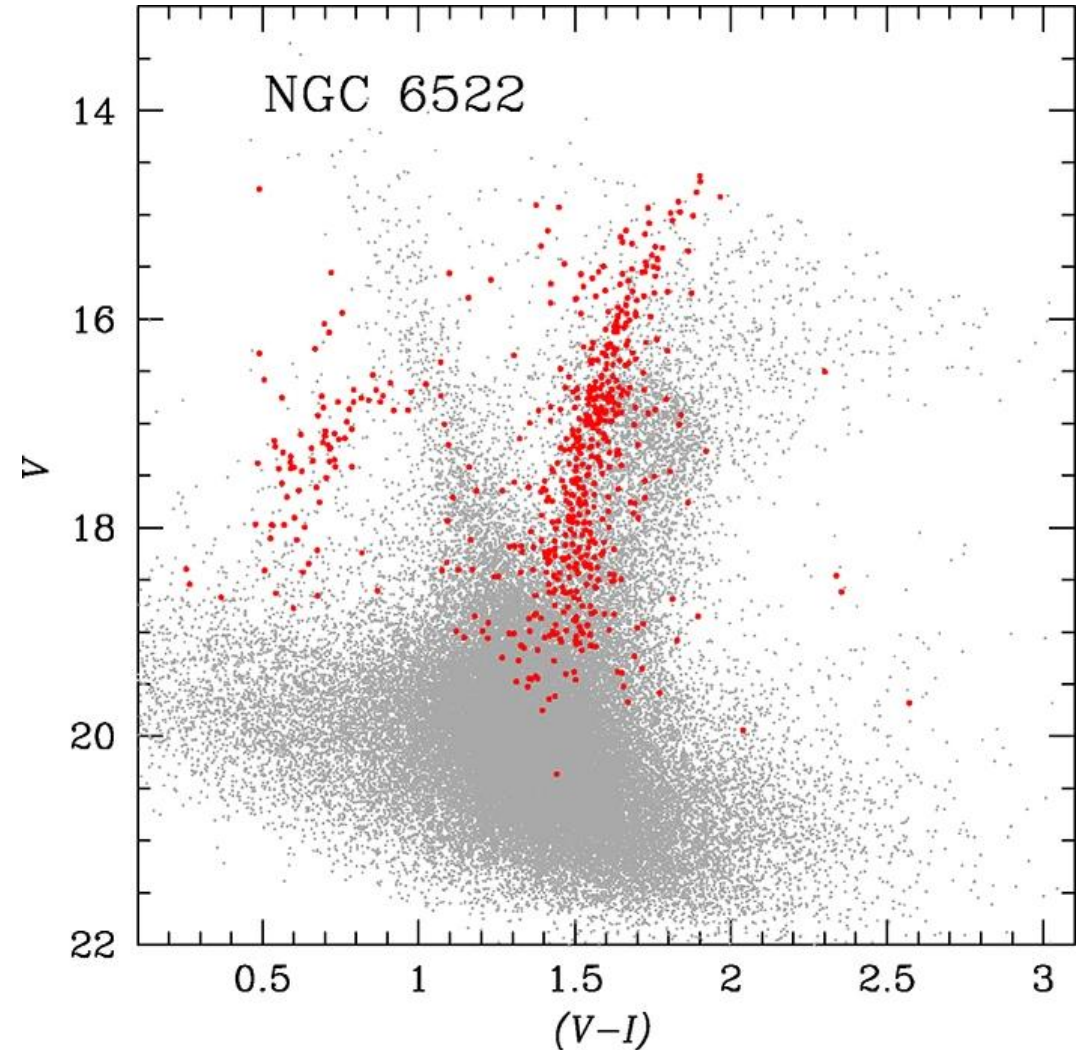
- Usually Euclidean distance is used
- Problematic when comparing features without a defined distance
 - E.g. morphology compared to magnitudes
- Other choices are possible

Scales and units

- KNN works on distances → the scale of the variables matters
 - Cannot compare differences in redshift with differences in stellar mass
 - In those cases, one features leads the classification
- Scale the features
 - Standardisation, mean = 0, std = 1
 - Normalisation, range [0,1]
- Units are extremely important for the same reason
 - Redshift vs. distance expressed in meters

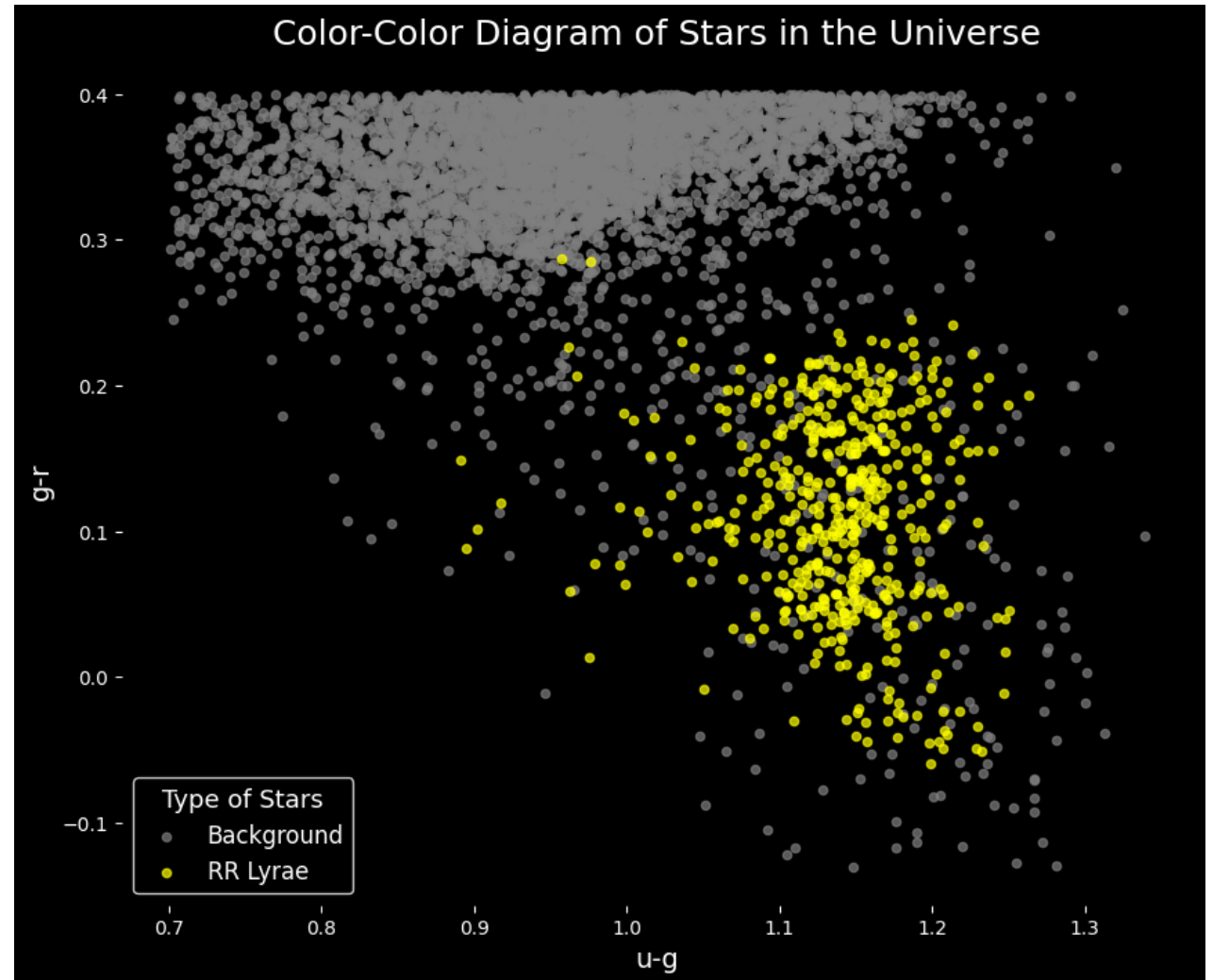
RR Lyrae

- Variable stars
- Standard candles
- How can we distinguish them from background stars with simple photometry?



RR Lyrae

- We can use a KNN using a dataset of four colours to classify new observations
- Hard to visualise in 4 dimensions
- Good for selecting candidates to follow-up



Pros

- Extremely simple and interpretable
- No assumption on the underlying distribution
- No training
- Versatile
- Can be interpreted in a probabilistic view

Cons

- Needs a large sample (like all non-parametric methods)
- Slow in high-dimension spaces and with many datapoints
- Curse of dimensionality
- Sensitive to noise



Decision Tree

Improving decision boundaries

Decision Trees – Intuition

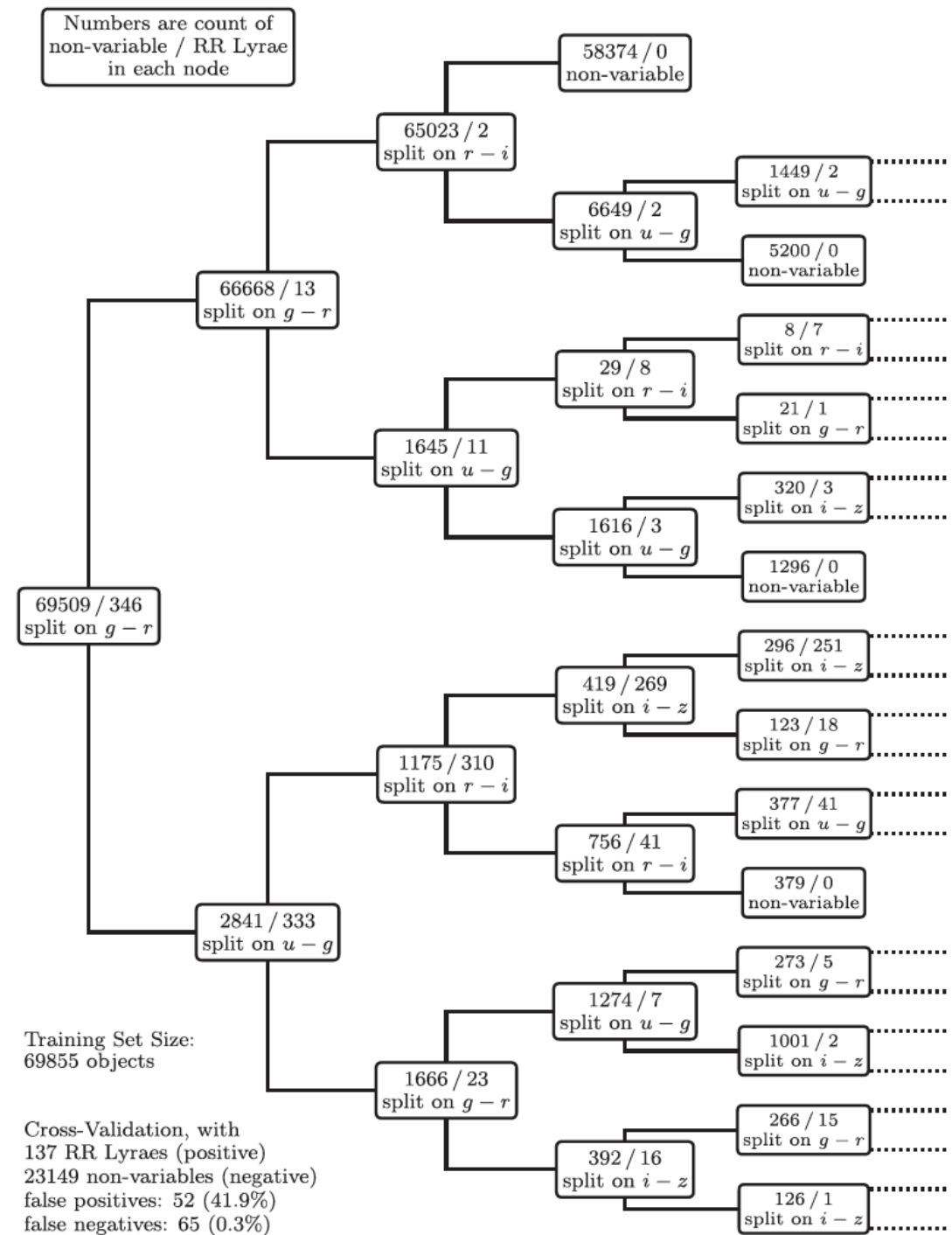
KNN classifies by distance to neighbours; Decision Trees classify by sequential rules

Segment the predictor space into a number of simple regions

- It is a **non-linear** method
- Recursively split the data based on their features
 - This translates in **branches** and end nodes, **leaves**
- To predict on a new observations, we use the training data in the region the new point ends up
 - Mean, weighted average, mode, etc.

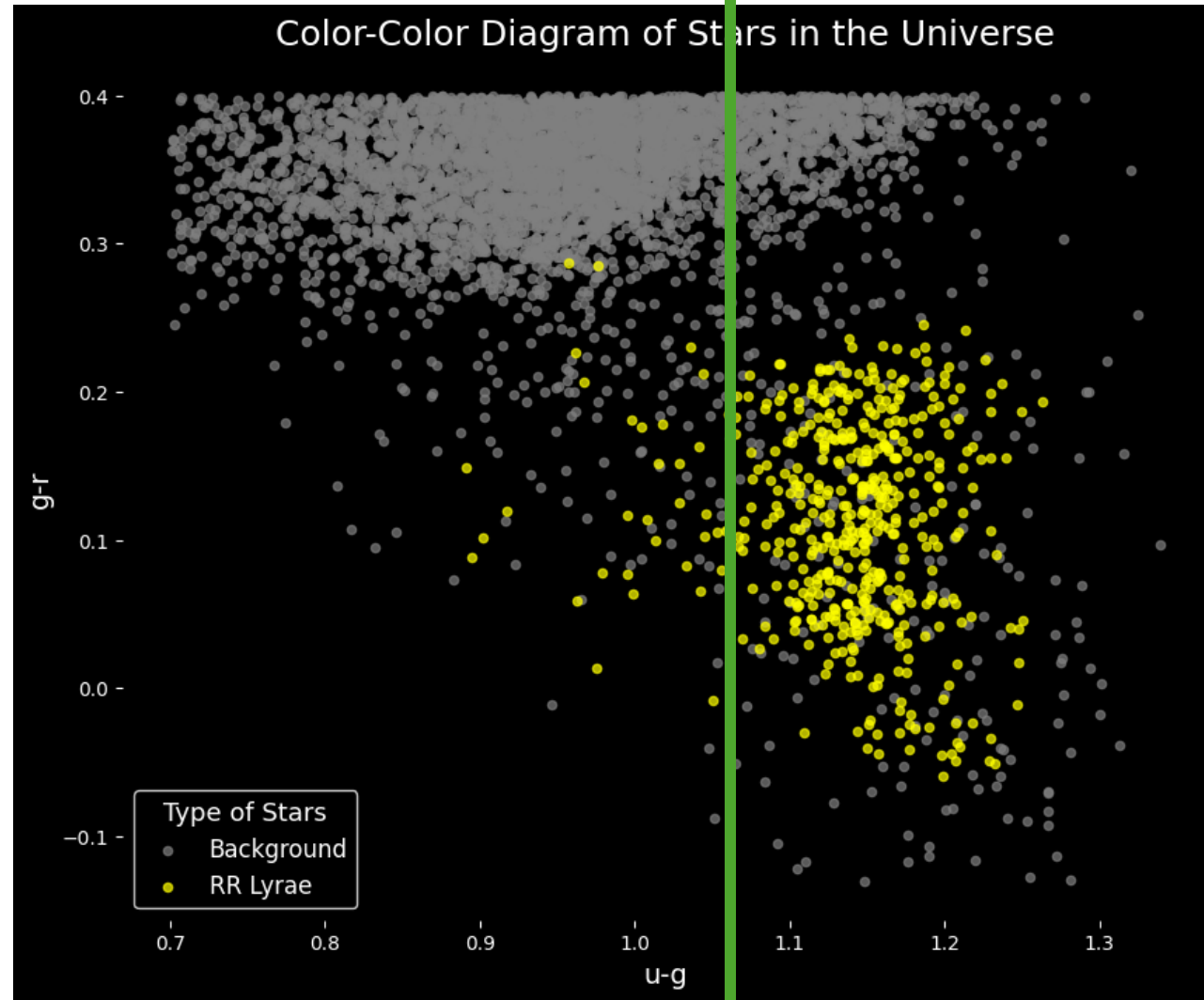
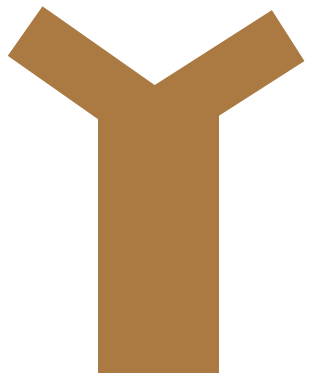
Intuition – Advantages

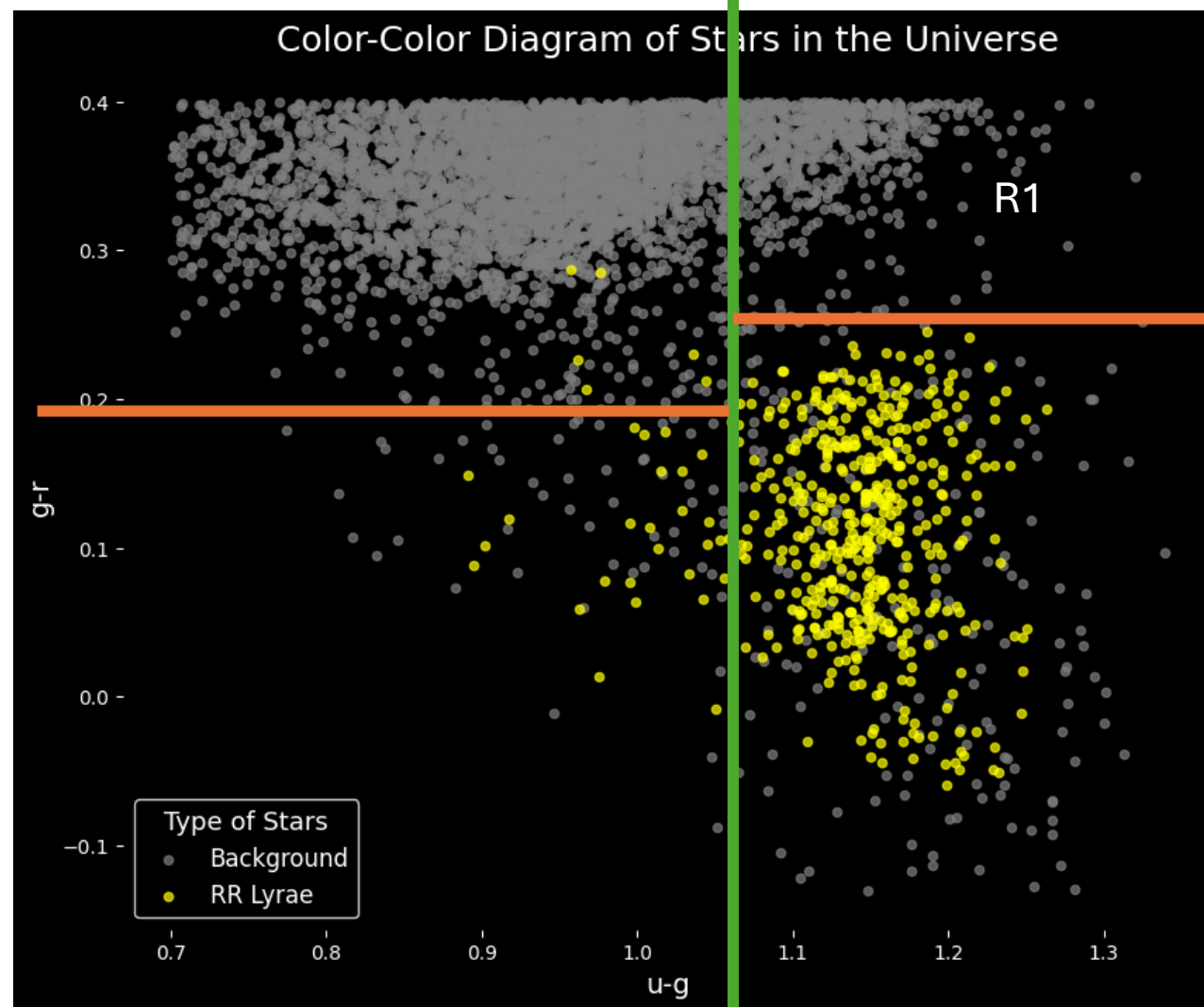
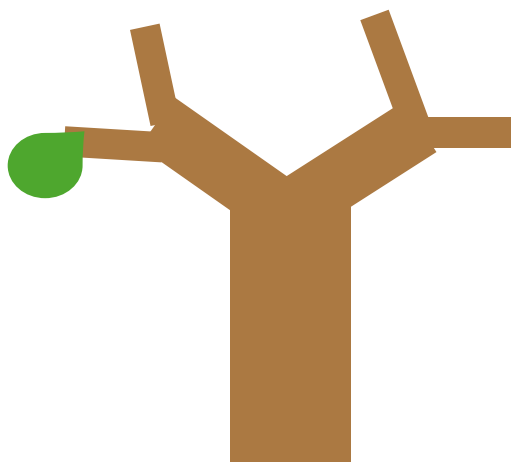
- Simple to understand
- Fully interpretable
 - You can literally read off the decision logic
 - It is a series of if statements

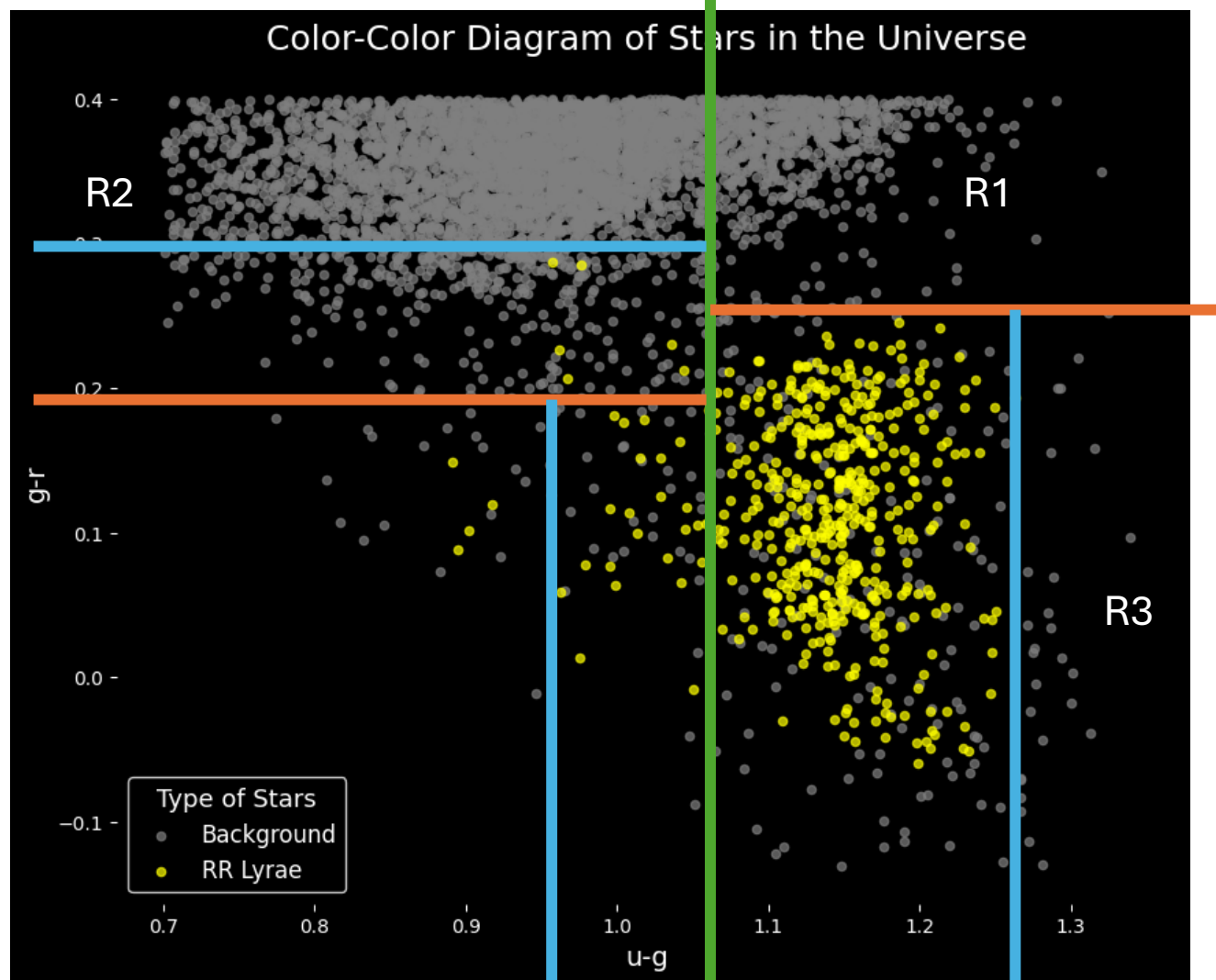
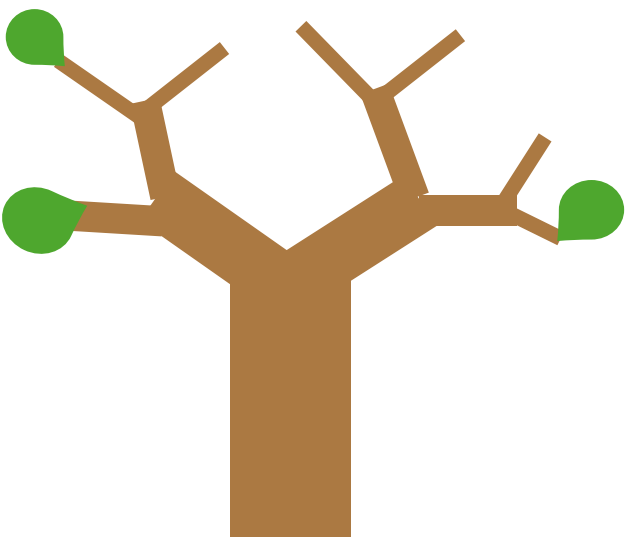


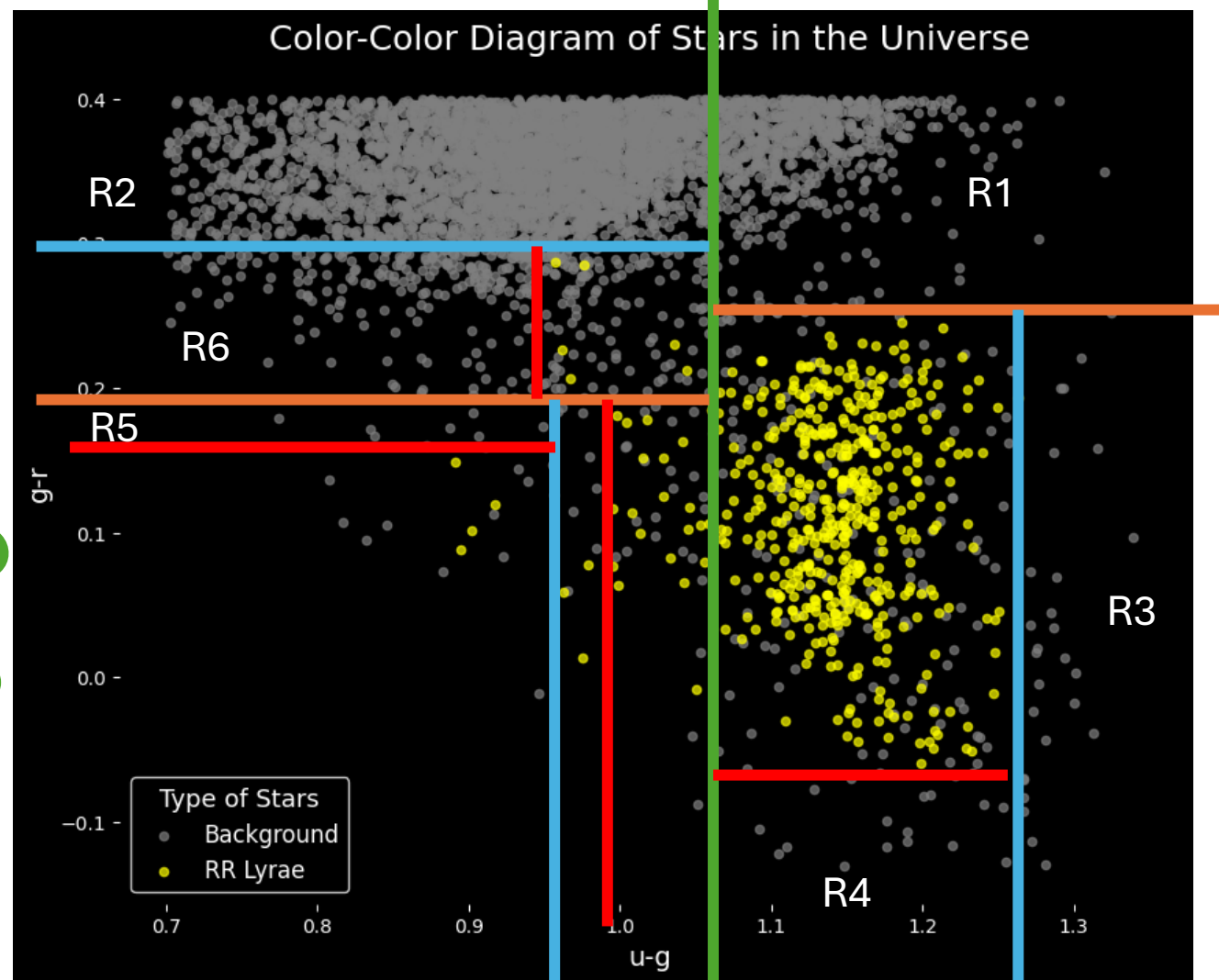
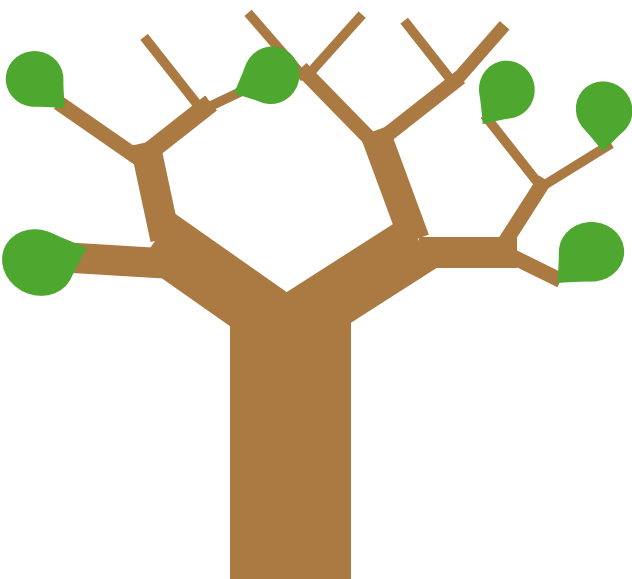
Building a decision tree

1. First node (**Root**): contains the entire data set
2. Subdivide the dataset into **two child nodes** or subsets (binary splitting)
 - Select a feature and a threshold
3. From the two new branches, we subdivided each child subset into other two subsets.
 - Usually, the data are split along one features at each level of the tree
4. This is repeated until a **stopping criteria** is reached (leaves)
 - The class fractions in the leaves determine the classification









Recap

- We **divide the space** into J non-overlapping regions, $R_1, R_2, \dots, R_J$
- For each new observation, we follow the branches and track in which region it falls, R_j . Then, we use the points in R_j to predict

→ The key aspect is understanding how to segment the space, how do we define the splits at each node?

We have to **learn** the optimal splitting

How splits are chosen

Goal: find $R_1, \dots, R_J$ regions that minimise a given error function

- We must select the feature and value on which to split the data

Let us start by considering a simple split criterion based on the information content or **entropy** of the data

$$E(x) = - \sum_i p_i(x) \ln(p_i(x))$$

$p_i(x)$ = probability of the i -th class in the training data

Entropy is very small if all $p_i(x)$ are either 0 or 1, i.e. *if the node is pure*

Information Gain = the reduction in entropy due to the partitioning of the data

IG = (Entropy before the split) – (Weighted sum of entropies after the split)

For a binary split, it is equal to:

$$IG(x|x_i) = E(x) - \sum_{i=0}^1 \frac{N_i}{N} E(x_i) = E(x) - \left(\frac{N_0}{N} E(x_0) + \frac{N_1}{N} E(x_1) \right)$$

- Maximise the IG at each split
- Finding the optimal decision boundary is computationally intractable
- **Greedy approach**

Greedy approach – much faster training

- Each feature is considered one at a time
 - The feature that provides the largest information gain is split
- Similarly, the value of the split is chosen.
 - We sort the data on feature j
 - Maximise the IG for a given split point s
- Being greedy comes at a cost: the algorithm finds the best split at each step, it does not look ahead for the “best overall series of splits”
 - **No guarantee we will get the optimal series of splits for our problem**

Other typical loss functions

- **Gini** coefficient (still related to the node purity)

$$G = \sum_i^k p_i (1 - p_i)$$

- Misclassification error/**Error rate** (mostly used for pruning)

$$ER = 1 - \max_i(p_i)$$

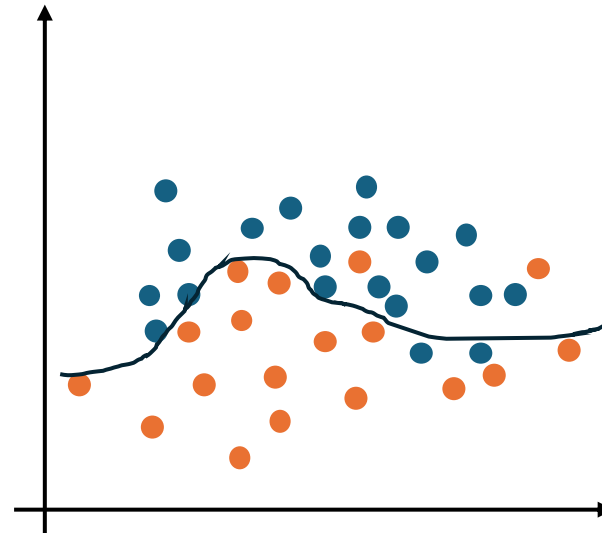
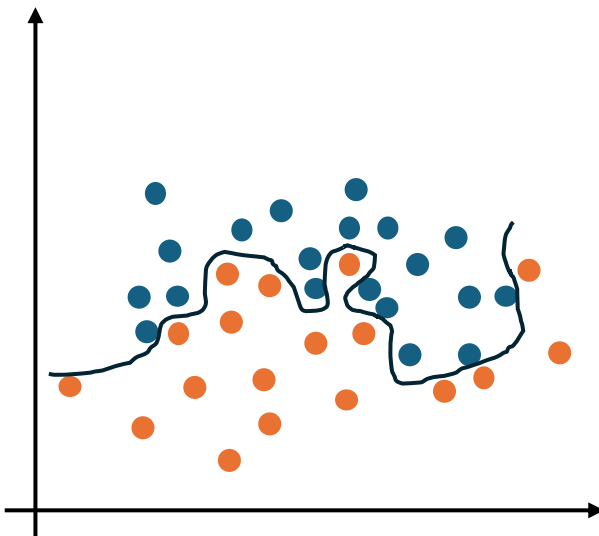
Stopping criteria

- **No criterion**: stop only when each leaf is pure (one-class)
 - Sometimes this means one object per leaf, extremely deep tree
- The user sets a **limit number** of objects **per leaf** (e.g. 5)
- **Pre-determined depth**
 - The depth controls the complexity of the tree and its overall performance
- No improvement in IG/Entropy
 - A new split does not produce any improvement in the IG/Entropy

Overfitting: the key problem

Without stopping criterion, a tree can grow until every leaf is pure

- Such a tree memorizes the training data: **overfits**
- At certain point, the classifier reflects only the noise
- The performance on unseen data is poor (low bias and very high variance)



Finding the optimal depth

Use **k-fold cross-validation**

- We can study the training vs. validation prediction error and find what is the optimal threshold after which the model overfits
- Classic bias-variance trade-off



Pruning

Let the tree grow fully, until there is one object per leaf

Remove the branches that do not improve the performance on the validation set (use one of the loss functions)

- Cost-complexity pruning: we cannot estimate this for all possible “subtrees”
- Use again a greedy approach → weakest link pruning

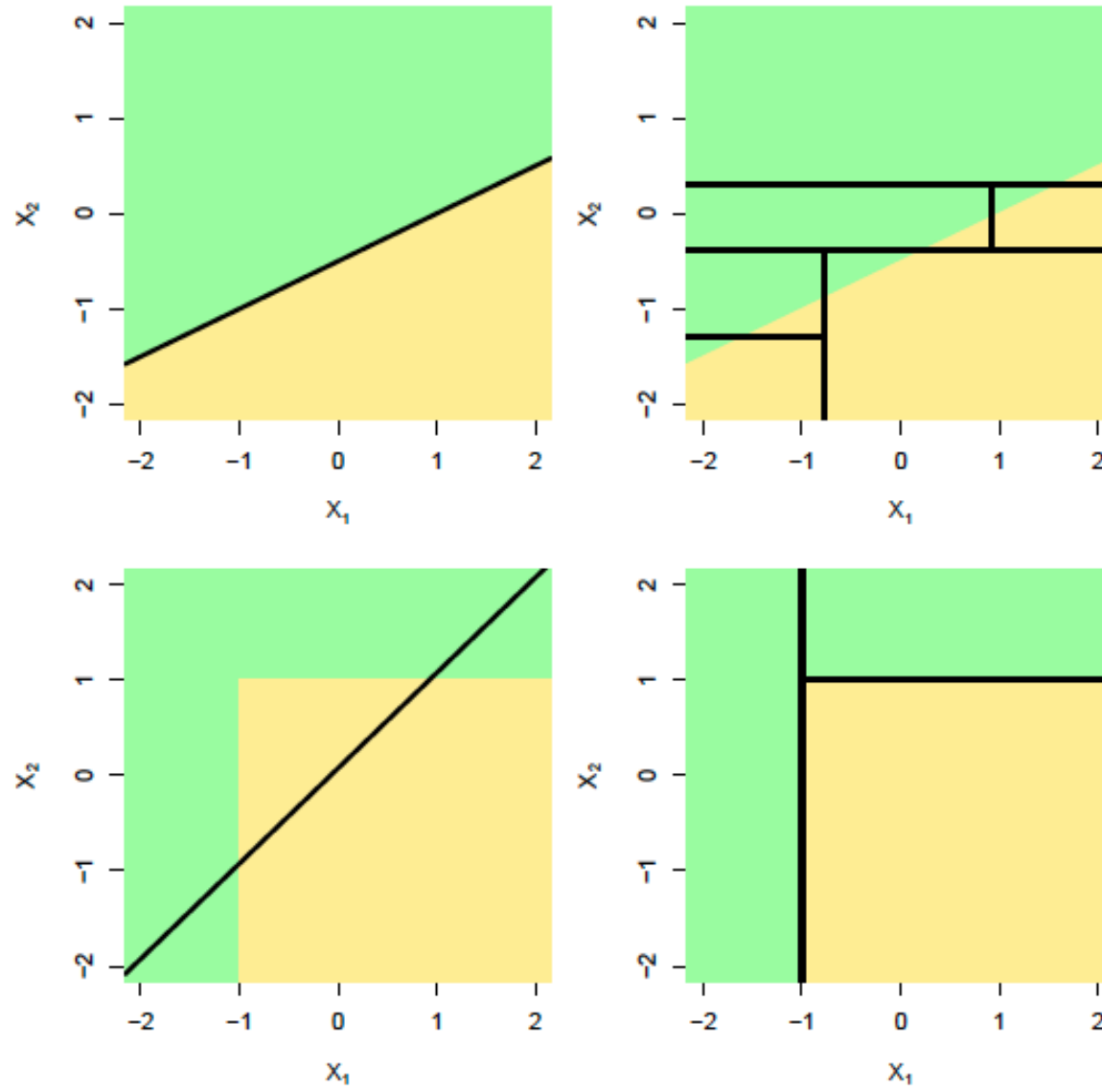
Strengths

- Simple, Fully interpretable: decisions read as if/then rules
- Captures nonlinear relationships with simple splits
 - Very effective for classes that are not linearly separable
- Handles mixed feature types easily
- No feature scaling needed
 - Threshold based, not distance based

Weaknesses

- Prone to overfit
 - Deep trees can memorize noise
- High variance: small changes in the training data can produce a very different tree
- Greedy splits
 - Locally optimal, not globally optimal tree
- Axis aligned splits
 - Struggles with diagonal decision boundaries. Needs a much deeper tree

Decision trees vs. Linear Models

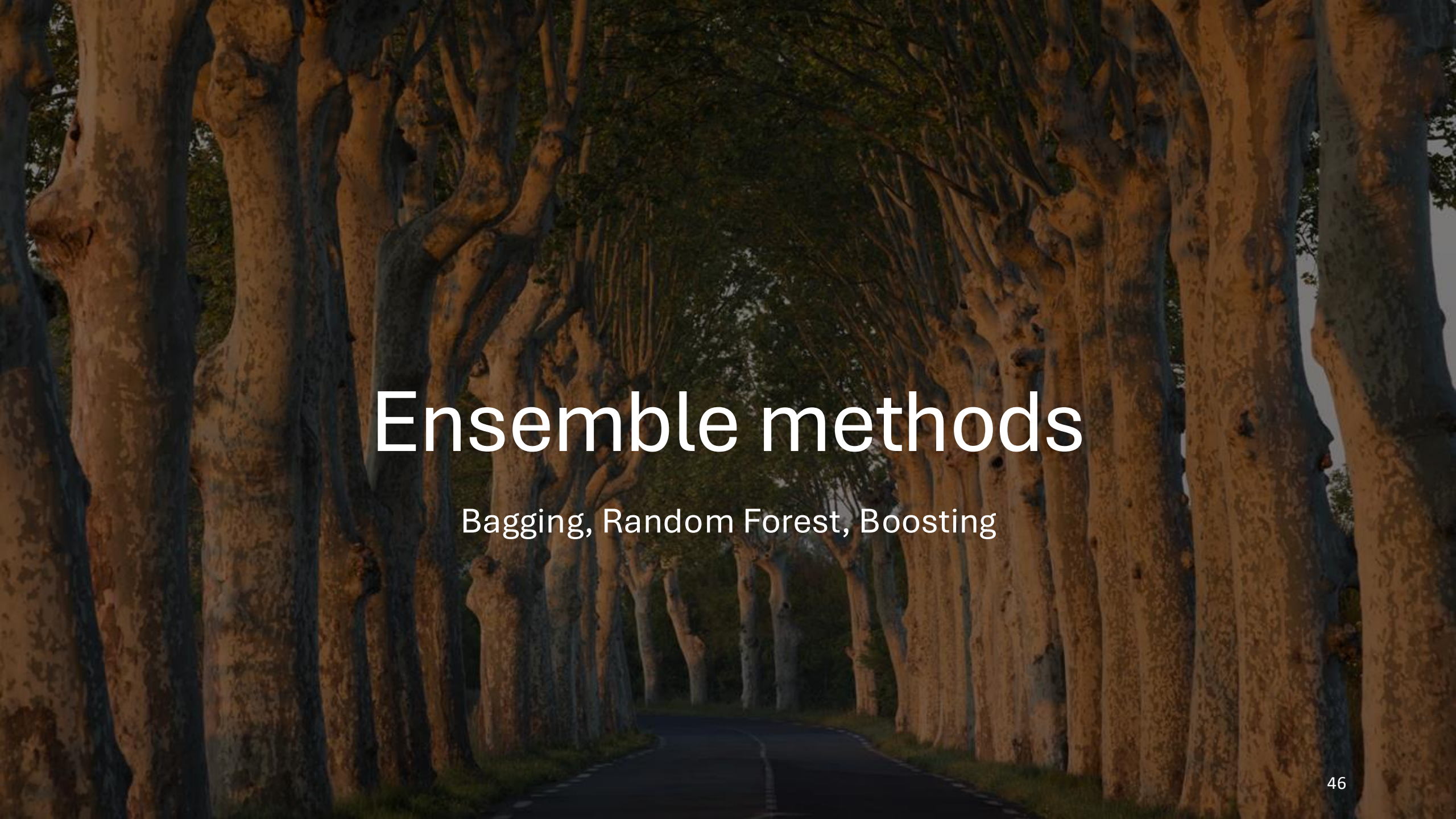


Decision trees – Take-away message

A single tree can be weak (**weak learner**) and not much better than other linear methods, but combining several trees can be extremely powerful

→ Ensemble methods:

- Bagging (o forest)
- Random Forest
- Boosting



Ensemble methods

Bagging, Random Forest, Boosting

Ensemble learning – Intuition

“Combining the outputs of multiple models through some kind of average” – **Wisdom of the crowd**

→ Combining single, **slightly different**, weak learners to obtain a single and more powerful model

Aggregation is a general-purpose approach used to deal with observations that have a very high variance

How to deal with observations with high variance

- You want to measure the size of a room, but your meter has a precision of $\pm 0.2\text{m}$
- You take multiple measurements of the room. Average the observations and obtain a better size estimate

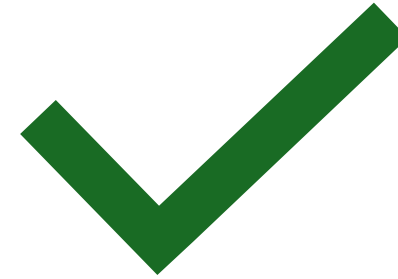
Given a set of n independent observations $Z_1, \dots, Z_n$, each with variance σ^2 , the variance of the mean $\bar{Z}$ of the observations is given by σ^2/n

Averaging a set of observations reduces variance

Ensemble learning – Intuition



Individual trees may overfit noise, but their errors tend to cancel out when averaged



This resolves most of the problems of decision trees

Bagging or bootstrap aggregation

Bootstrap the training sample → obtain slightly different training datasets for each tree

Bootstrap = take repeated samples (B), equally sized, from the single, initial, training sample. Usually done with *replacement*, hence each sample has the same size as the original training set

Build B trees, each one trained on a different b bootstrapped samples

Typically, $\sim 1/3$ of the training sample is left out from each bootstrapped sample – this can be useful for error estimation

Bagging or bootstrap aggregation

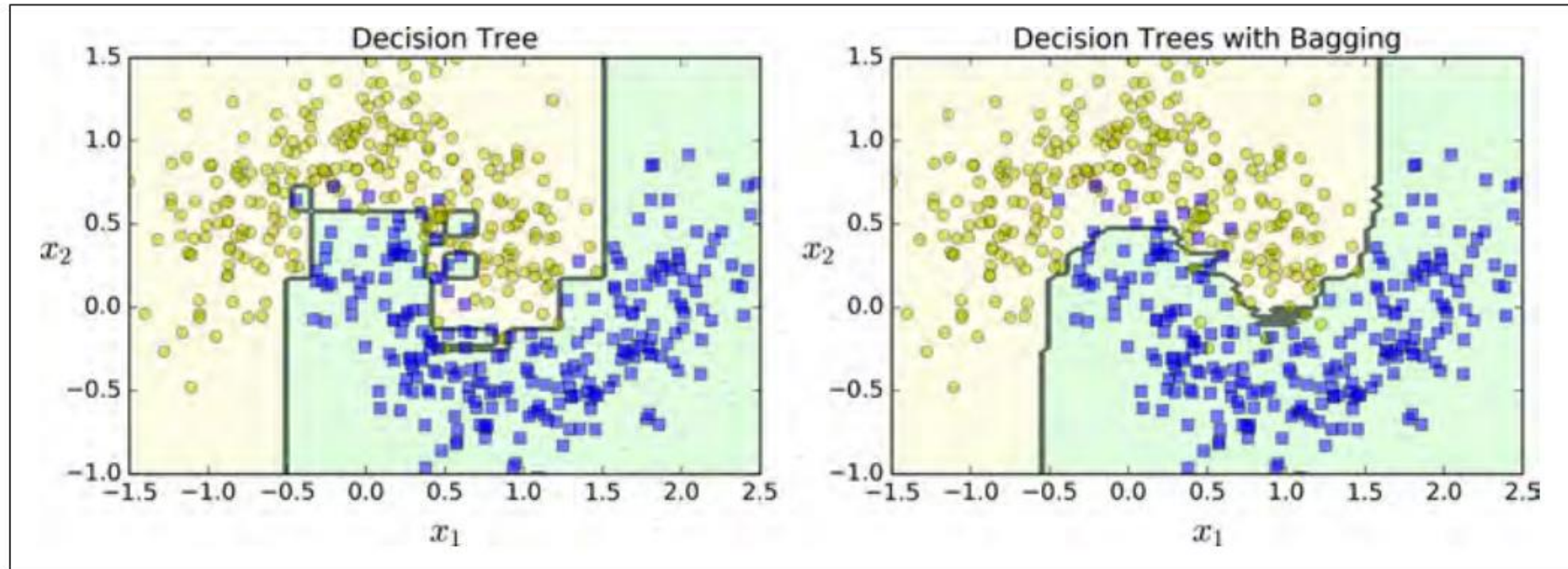
Each tree is a model $f_b(x)$

Combine the individual trees, averaging their results

$$f(x) = \frac{1}{B} \sum_{b=1}^B f_b(x)$$

For each test observation we record the class predicted by each tree and take a **majority vote**

Bagging or bootstrap aggregation



Bagging issues

But these trees are **correlated**

- If there is one very strong feature in the dataset, then most (if not all) bagged trees will use this strong feature in the top split.

→ Consequentially, all trees will be quite similar to each other.
Hence, the predictions from all trees will be highly correlated.

Bagging issues

- Therefore, one of the basic hypothesis (independence) is not entirely true
 - The variance in this case does not really go down as σ^2/n
- If we **decorrelate the trees**, we can get much lower variance

We want an ensemble of trees that are sufficiently different from each other.

Random Forest – Intuition

- We initialise multiple decision trees on bootstrapped training samples
- When building these trees, each time a split is considered, only a **random selection of features** is considered from all possible features
- Random feature selection decorrelates the trees, making the averaging really effective

Random Forest

- We have n trees
- Select a number m of random features to use, where $m \leq p$
 - If $m = p$, it is bagging

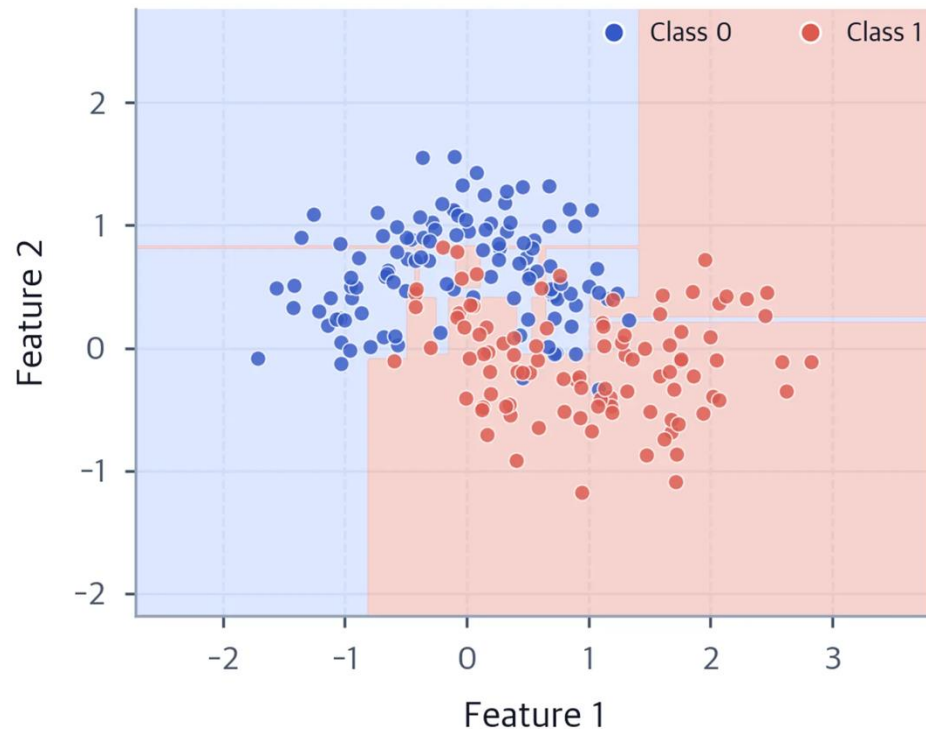
Keeping m small, we can control the complexity of the model and prevent overfitting

Prediction: each tree “votes” for a class, majority vote wins (or average predicted probabilities)

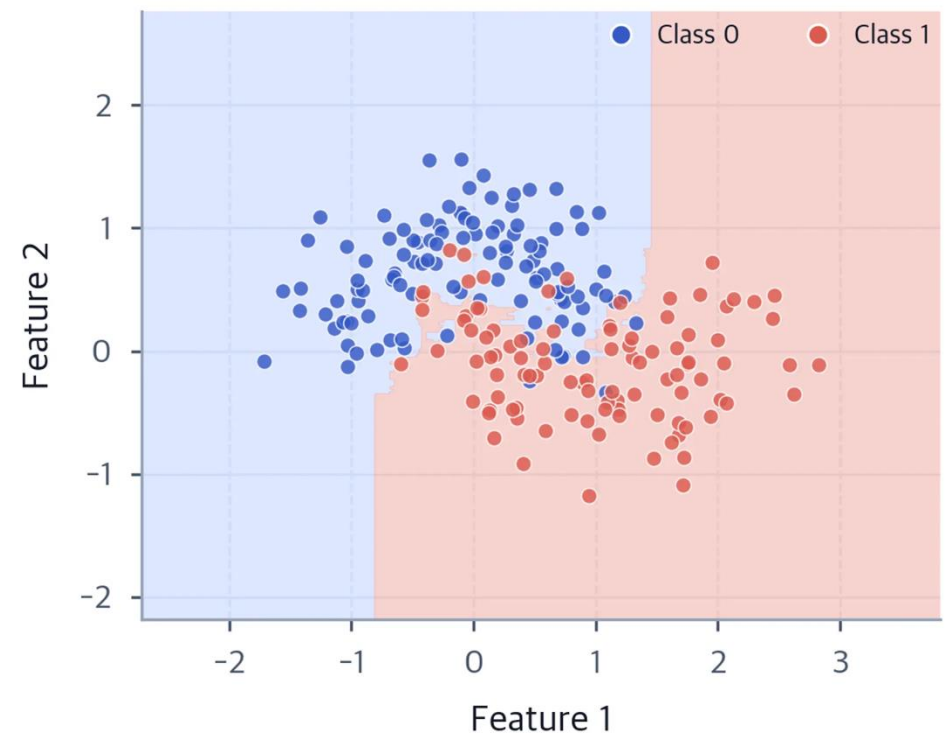
Random Forest resolves

1. Overfitting problem of deep trees
2. The fact that axis-aligned partitioning of the data does not really reflect smoother decision boundaries

Individual Decision Tree
(Prone to Overfitting)



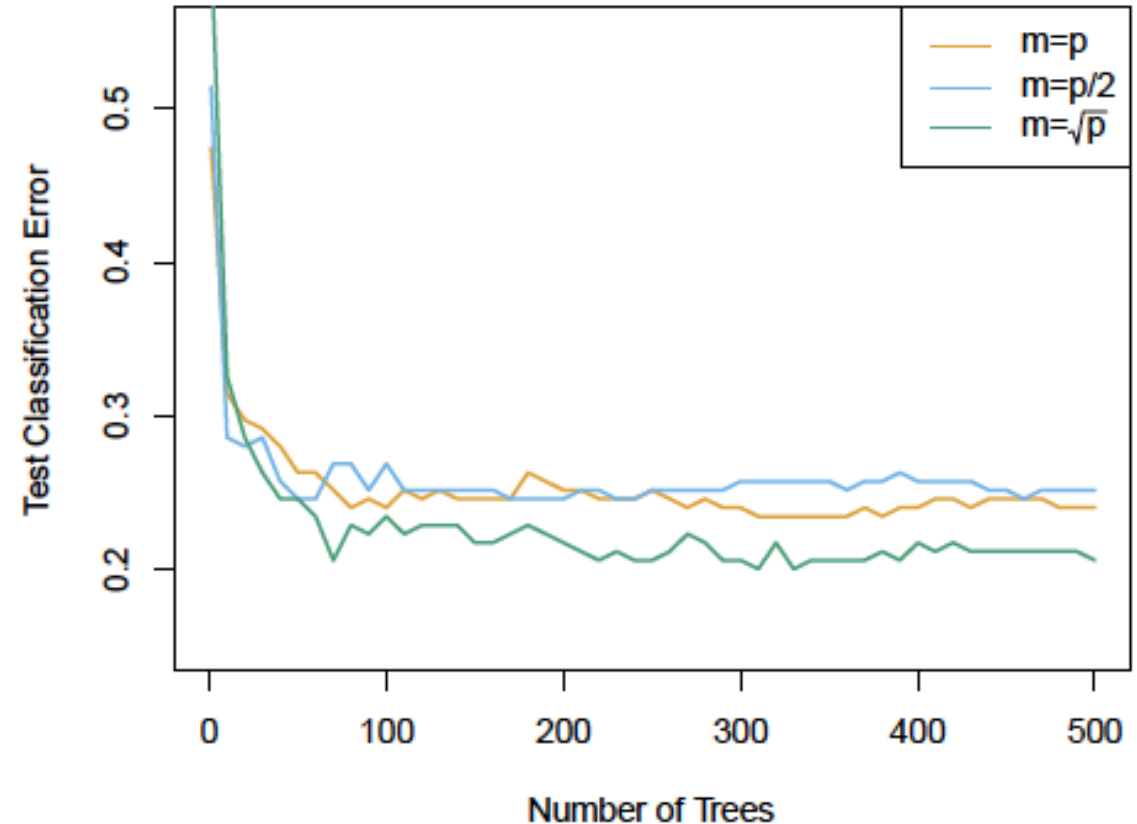
Random Forest Ensemble
(More Robust)



Finetune a Random Forest

Hyper-parameters

- Number of trees
 - Depth of each tree (or stopping criterion)
 - Number of features to use
- Cross-validation can be used to find the optimal values
- Increase the number n of trees until the cross-validation error plateaus
 - The number of features is usually set as $m \sim \sqrt{p}$



Out-of-bag (OOB) observations

The bootstrapping procedure gives a validation set for free

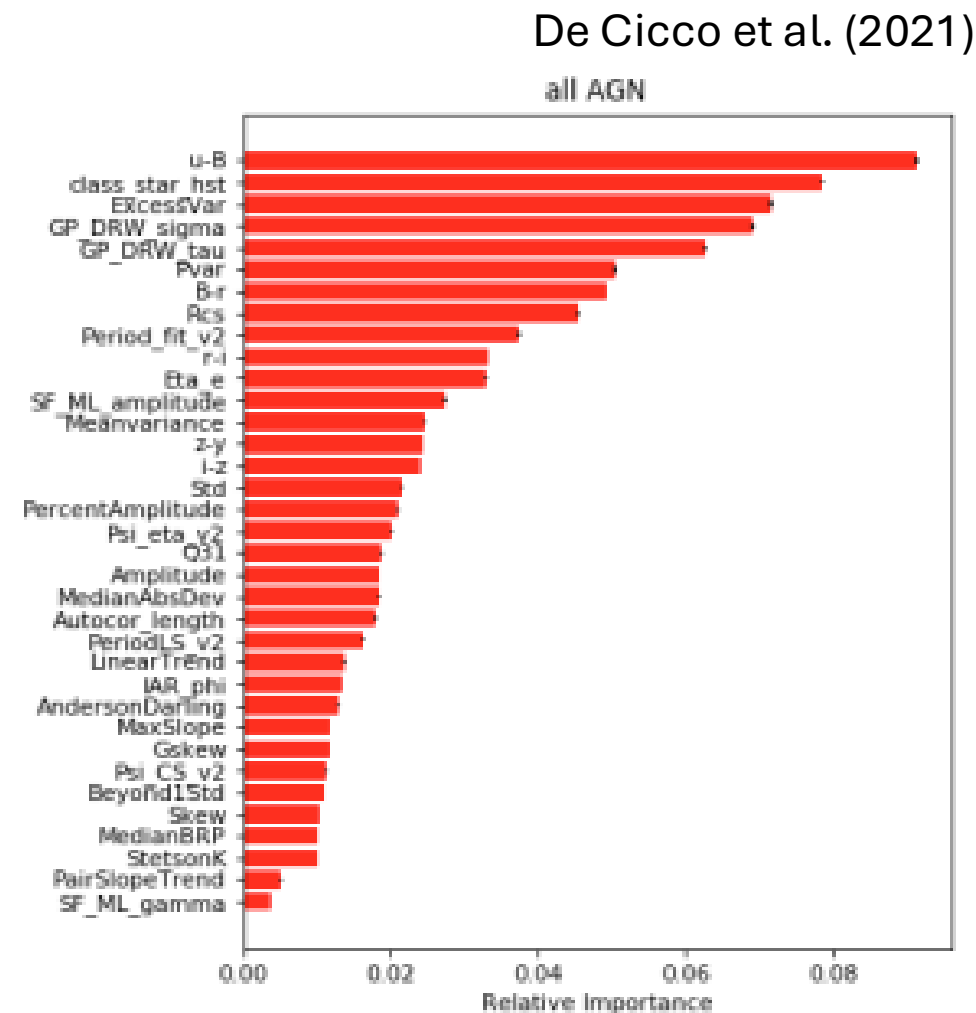
- Recall that $\sim 1/3$ of the training sample is not seen by trees
- Trees that did not see this **left-out-sample**, can use it for making predictions
 - Unbiased **error estimate**
 - Each tree uses a different “validation set”
 - It can be shown that this is equivalent to cross-validation, if large enough

Interpretability problem

- Bagging and Random Forest improve prediction accuracy at the expenses of interpretability
- Now we cannot visualise the logic classification scheme

Feature importance

- We record the total amount of times a feature is used successfully (i.e. every time IG increases), averaged for all trees
- Information about which features actually drive the classification
 - Useful to inform next observations
 - For example, z-band observations reveal to be the least informative. I save some time for the next observations



Random Forest in Astronomy

- Often used for regression: estimate photometric redshifts, stellar mass, etc.
- Used for spectral data classification: galaxy/star/QSO problem
- Find optically variable AGN in images

Property	Random Forest	Decision Tree
Interpretability	No easy to interpret directly. We can use feature importance	Highly interpretable
Overfitting	Due to ensemble averaging, less prone to overfitting	Very easy to overfit with deep trees
Training time	It takes longer because multiple trees are constructed. Also requires larger memory	Much faster, single tree. Low memory need
Predictive time	Slower, every tree needs to predict and then average	Faster
Stability	Since average is taken, it is very stable to change	Very sensitive to variation
Outliers	Very robust to outliers	Very susceptible to outliers
Performance evaluation	OOB error for free	Needs a separate dataset

Can we improve this concept further?

- Random Forest is a collection of weak (*stupid*) learners – can we use better individual learners?
- Each tree is trained in parallel on different child samples
- What if we proceed in **series**? Building each tree sequentially, using the information from the previous tree?
 - Each new tree can try to fix the mistakes of the previous ones

Boosting – Intuition

- Trees are grown **sequentially**, each one takes the information from the previous one and corrects that classification
- Idea to reweight the data based on the incorrect classifications of the previous tree

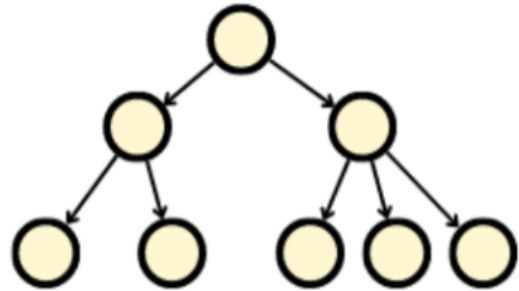
Instead of reducing the overall variance via averaging (RF), we iteratively reduce the bias of each tree focusing on hard cases

Boosting – Intuition

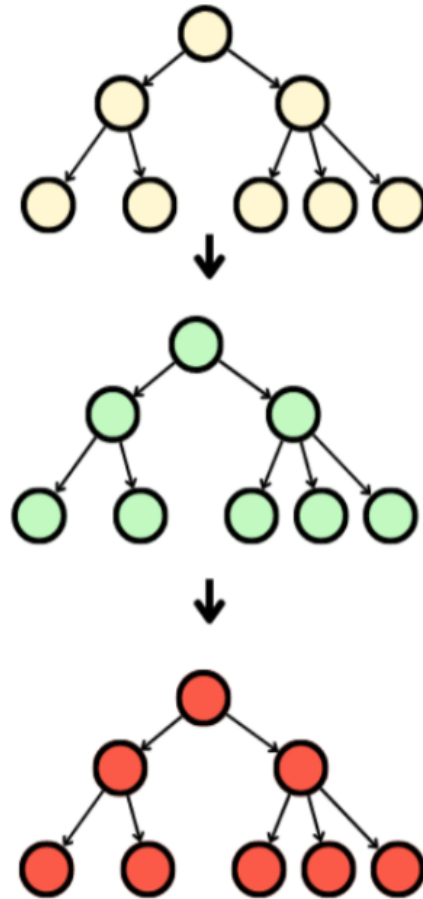
- Ensemble of trees, where each new tree is better than the previous
- We combine the models using a **weighted average**
- Best trees have larger “voting” weight for the final prediction

Slow learning: we slowly improve the tree in areas where it does not perform well

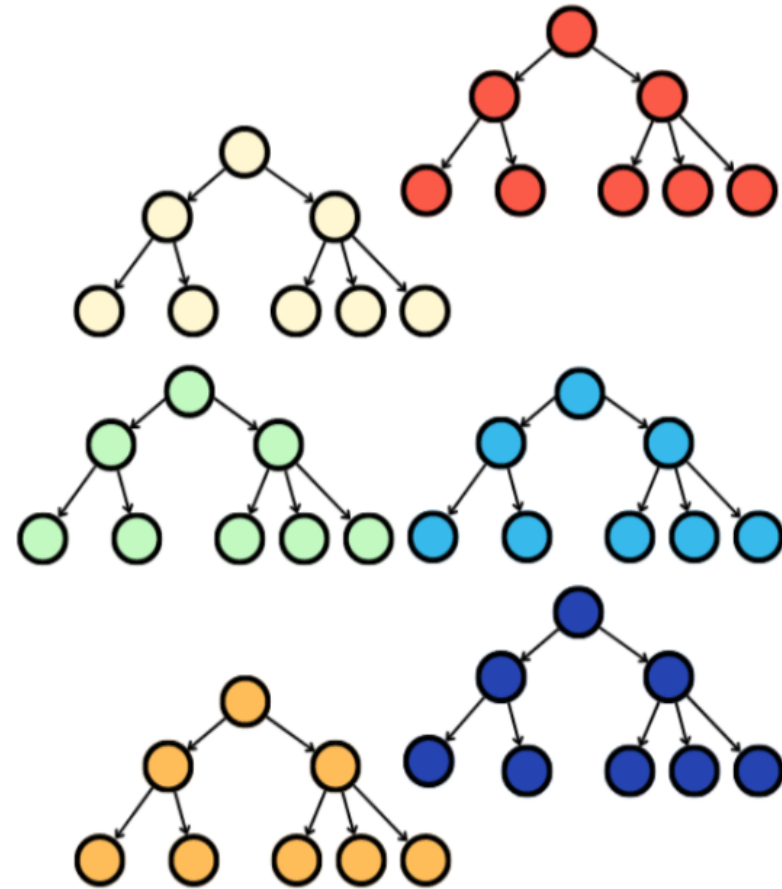
Decision Tree



Boosted Tree



Random Forest



Adaptive Boosting (**AdaBoost**)

Starting point: weak classifier $h(x)$

Goal: **strong classifier** $f(x)$ that is our ensemble boosted tree

$$f(x) = \sum_m^K \theta_m h_m(x)$$

Where m indicates the iteration and θ_m is the weight of the m -th classifier

Adaptive Boosting (**AdaBoost**)

We have a training dataset, x , with associated labels y

1. First iteration we assign uniform weights to all points. If $w_m(x_i)$ is the weight for the x_i point, then $w_0(x_i) = 1/N$ for all i
2. Apply the weak classifier $h(x) \rightarrow$ predictions $h_0(x_i)$
3. Estimate classification error at that iteration

$$e_m = \sum_{i=0}^N w_m(x_i) \cdot \delta(h_m(x_i) \neq y_i)$$

Adaptive Boosting (**AdaBoost**)

4. From the error we can estimate the weight of the weak classifier at iteration m as

$$\theta_m = \frac{1}{2} \log \left(\frac{1 - e_m}{e_m} \right)$$

5. Then we need to update the data weights for next iteration, $m + 1$

$$w_{m+1}(x_i) = w_m(x_i) \times \begin{cases} e^{-\theta_m} & \text{if } h_m(x_i) = y_i \\ e^{\theta_m} & \text{if } h_m(x_i) \neq y_i \end{cases}$$

Misclassified data will get larger weights

Adaptive Boosting (**AdaBoost**)

As the total misclassification error, e_m , decreases, the weight of that iteration in the final classifier increases.

Example:

Iteration 1) $e_m = 0.3$ then $\theta_m = 0.18$

Iteration 2) $e_m = 0.1$ then $\theta_m = 0.48$

Iteration 3) $e_m = 0.05$ then $\theta_m = 0.63$

Iteration 4) $e_m = 0.01$ then $\theta_m = 0.99$

Gradient Boosting

- Introduced to speed up boosting
- Approximate a steep descent criterion after each simple evaluation
- Each new tree is fit to the gradient of the loss function
 - What direction should predictions move to reduce error

XGBoost – Extreme Gradient Boosting

Most popular tree boosting model

- Efficient
 - Second-order gradient information (uses also the Hessian for more precise updates)
- Regularized
 - Penalises tree complexity and helps preventing overfitting
- Highly optimised
 - Parallelised tree construction, can handle missing values
 - Built-in cross-validation

Boosting hyperparameters

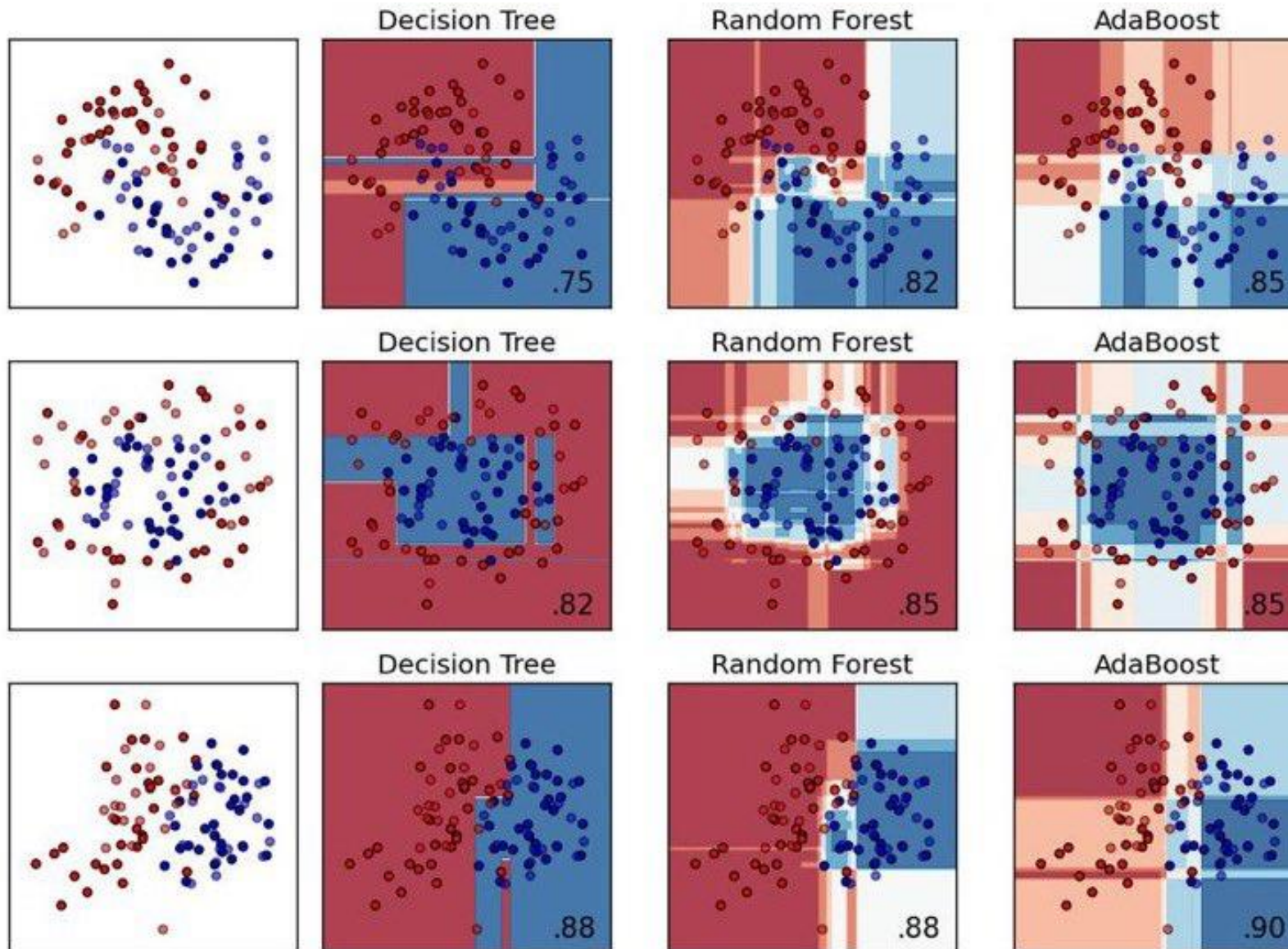
- Number of trees / boosting rounds → Risk of overfitting
- Learning rate (shrinkage) λ
 - Controls the rate at which boosting learns
 - Low values → more rounds
- Max depth / stopping criteria for each tree
- Early stopping if performance does not improve on validation
- Fraction of features
- Fraction of sample to use
- Regularization (L1, L2) terms if we want to control overfitting

Strengths

- Better performance compared to RF
 - Usually the best on tabular data
- Can handle missing features
- Prevent overfitting
- Feature importance

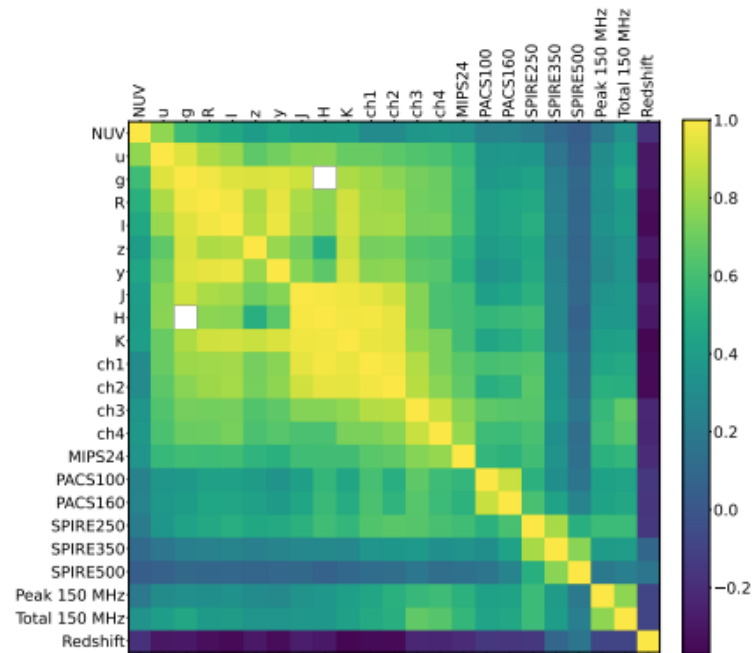
Weaknesses

- Complex to tune
 - More hyperparameters
- Slower than RF
- Needs more data
 - More computational power

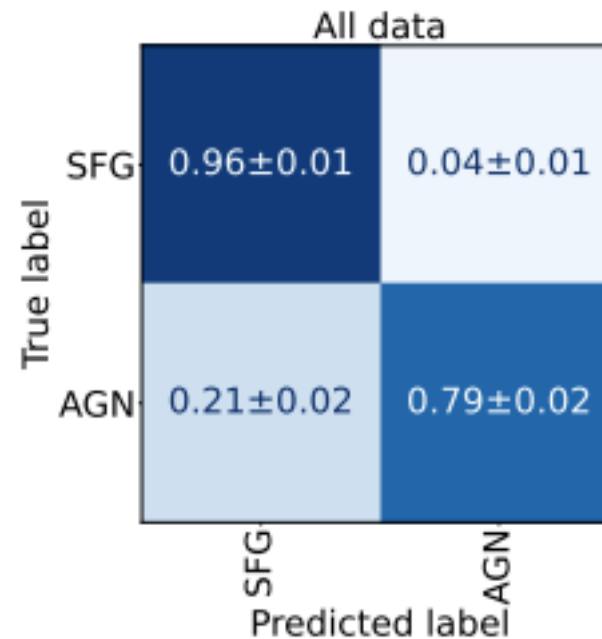


Astronomy example: AGN-SFG classifier for radio surveys (Karsten et al. 2023)

- Extensive list of features
- Light Gradient Boosting
- Log-loss function



	Search space	Optimal value
num_leaves	10–50	46
learning_rate	0.001–0.8	0.06369
min_data_in_leaf	1–20	1
colsample_bytree	0.1–1	0.5468
reg_alpha	0–5	2.619
reg_lambda	0–10	7.873



Recap



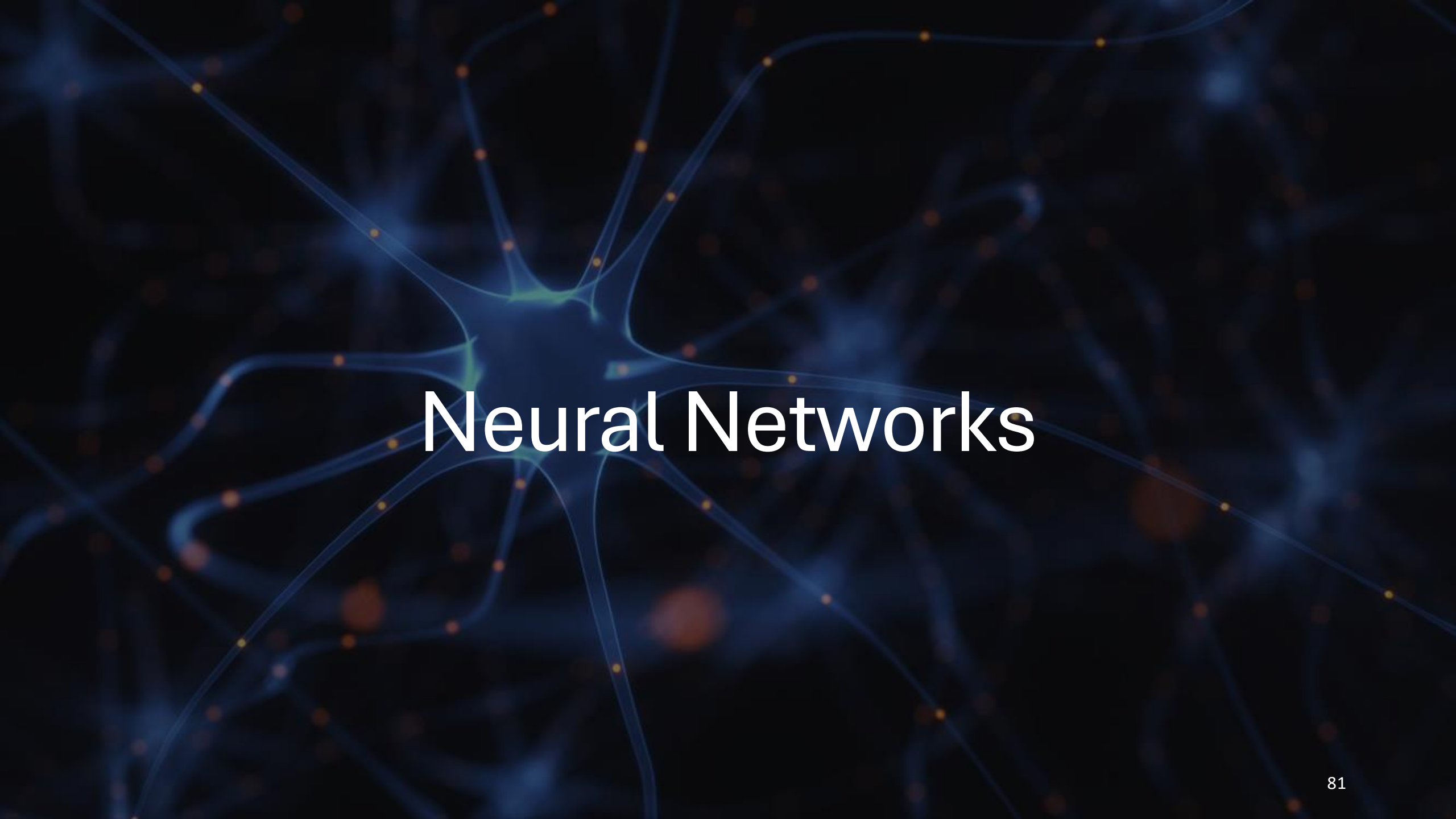


1. Easiest approach: similar things behave similarly (KNN)
2. Simple interpretable decision trees (logic flowchart)
3. Parallel averaging many different trees, trained on randomly different training set (RF) – Wisdom of the crowd
4. Sequentially correcting the mistakes of each tree – Boosting

Overall comparison

Aspect	KNN	Decision Tree	Random Forest	Boosting
Core idea	Classify by nearest neighbours	Sequential rule-based splits	Many bagged trees, averaged	Many trees, correcting previous errors
Training	None	Fast, single pass	Parallel (independent)	Sequential (each tree depends on the last)
Inference	Slow	Fast	Fast	Fast
Interpretability	Low	High	Medium	Medium
Bias/variance	Low bias, high variance	Low bias, high variance	Low variance	Low bias and variance
Overfitting risk	High (for small K)	High	Low	Moderate
Hyperparameter	Just one	A few	More than a few	Many

- Classical methods work well on engineered features
- What if the input is raw pixel data? What if we want to use just the observations as they are?



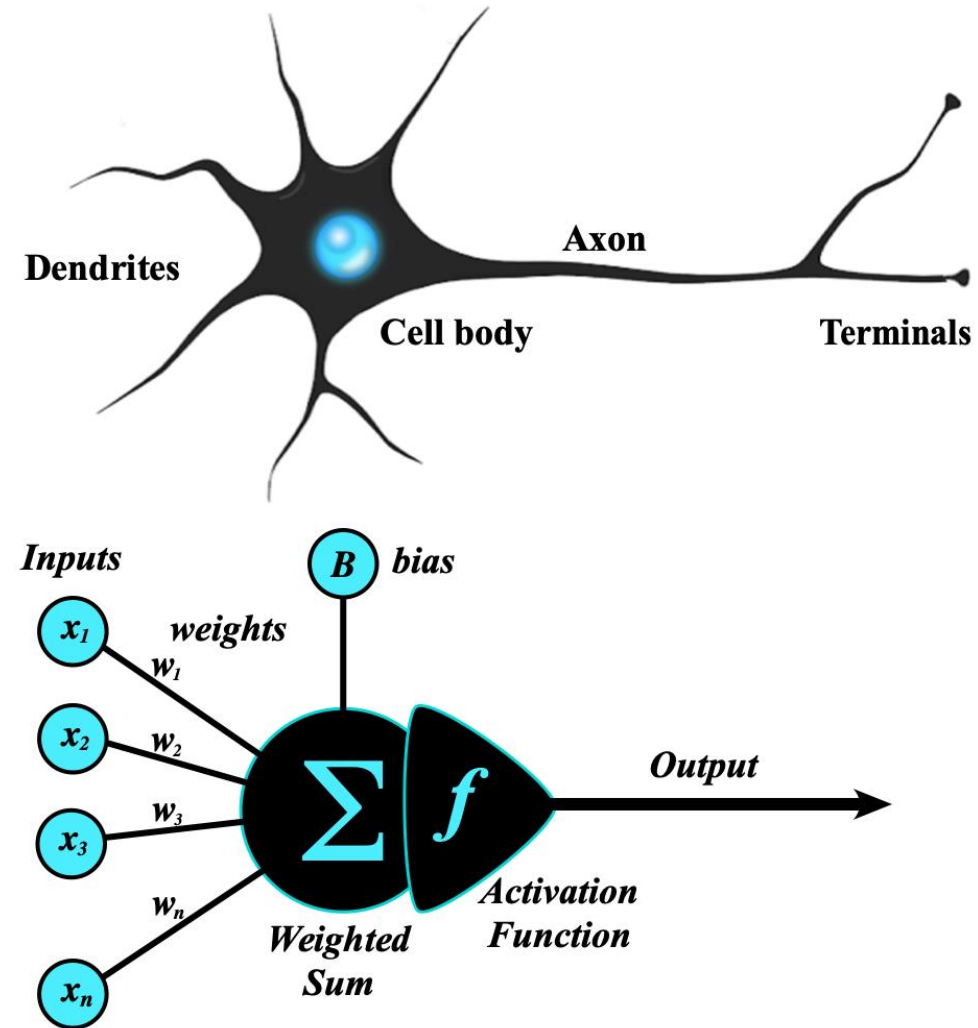
Neural Networks

Neural Networks – Intuition

The concepts behind NN are inspired by the structure and function of the brain

The computational unit is the **neuron**

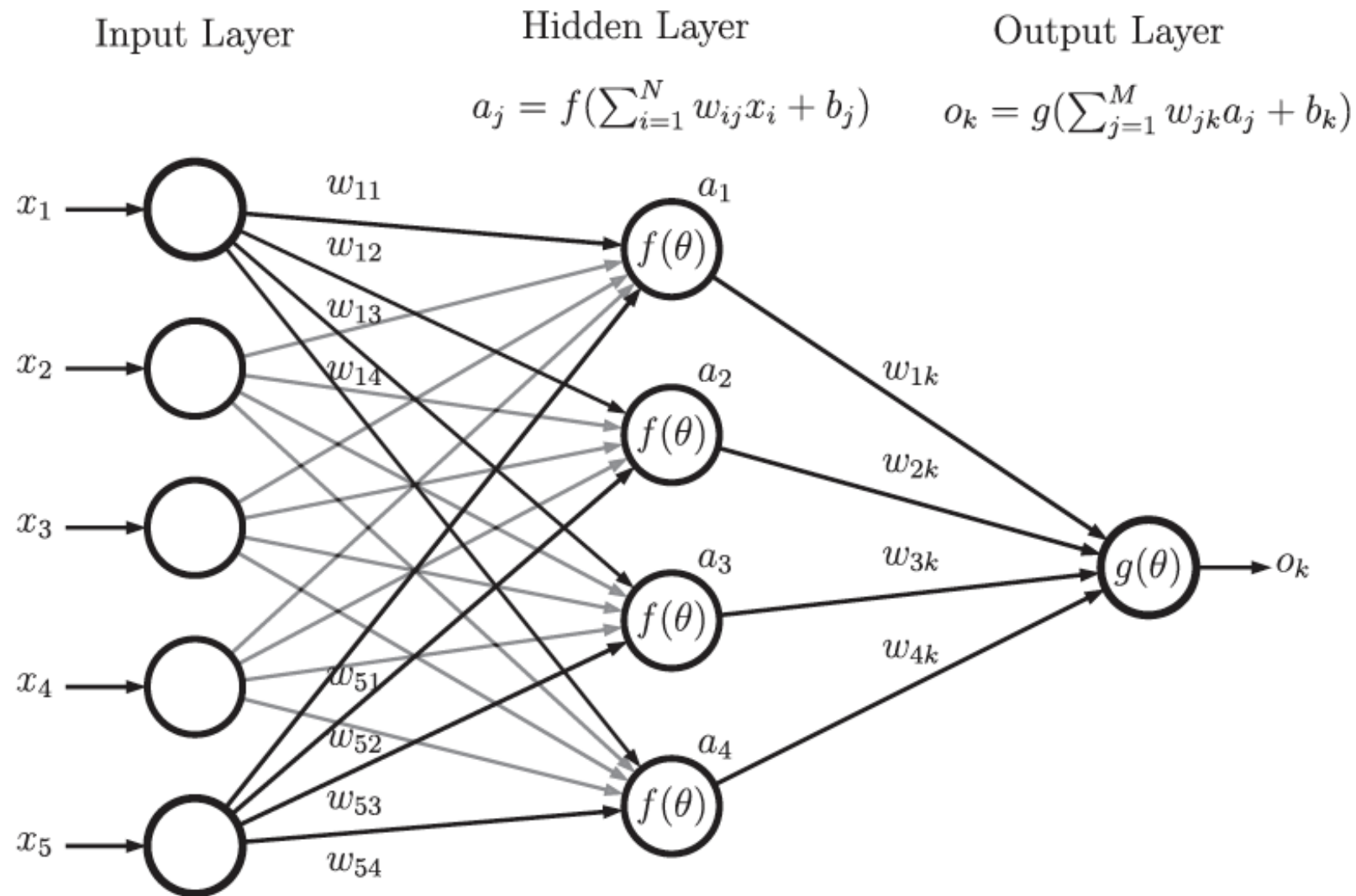
- A series of inputs reach the nucleus from the dendrites
- The nucleus operates on these inputs and generates an output
- The output is transmitted along the axon to other neurons or the user



Multi-Layer Perceptron

- Connect multiple neurons
- Form layers of neurons that receive inputs and produce outputs
 - Each neuron generates its own output
- Connect different layers
 - The connections have weights to adjust

Simple feed-forward NN



Glossary

- **Hidden layers**: layers of neurons between the inputs and outputs
- **Feed forward**: neurons only connect to subsequent layers
- **Fully-connected**: each neuron of a layer is connected to all neurons of the subsequent layer
- **Activation function**: the function of each neuron that operates on the inputs and produces an output for the next layer

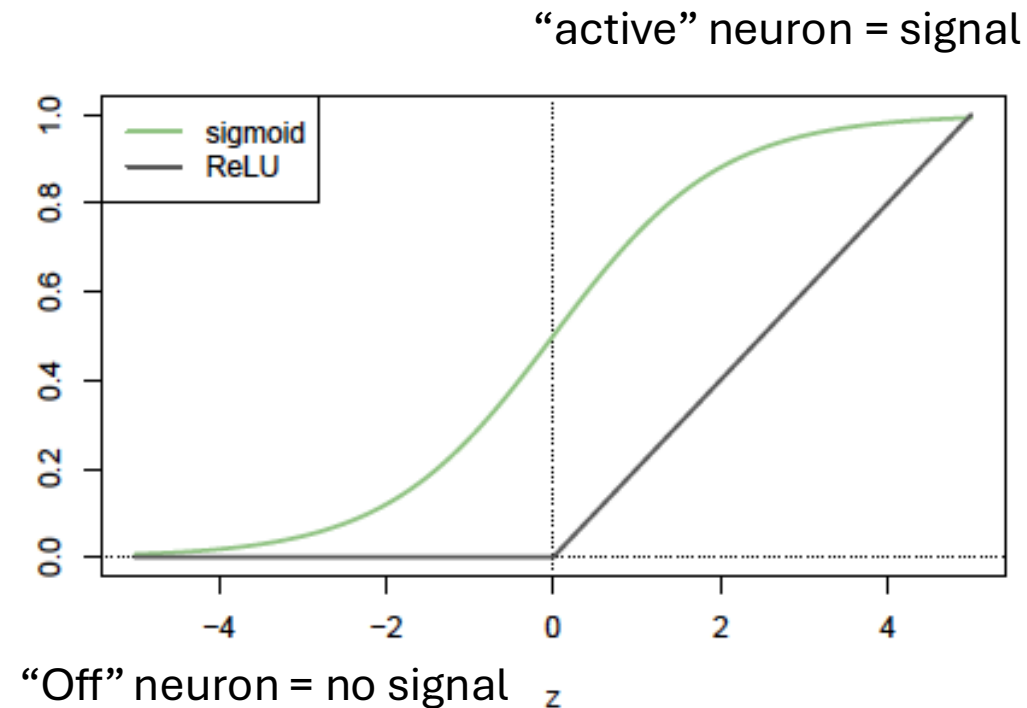
Activation function

$$A_j = f\left(\sum_i w_{ij} x_i + b_j\right)$$

Where b_j is the bias term. It modulates the activation level

- Sigmoid $A(z) = \frac{e^z}{1+e^z}$
- Rectified linear unit (ReLU)

$$A(z) = \begin{cases} 0 & z < 0 \\ z & z \geq 0 \end{cases}$$



Activation Function

The non-linearity in the activation function is essential

- If the activation function is linear, the neural networks becomes only a single, large, linear model
- Non linearities allows the model to capture complex structures

Output layer

- As the other layers, acts on the weighted sum of its inputs
- Single output for binary classification, or multiple outputs for multiple classes

$$O_k = g \left(\sum_j w_{jk} A_j + b_k \right) = g \left(\sum_j w_{jk} f \left(\sum_i w_{ij} x_i + b_j \right) + b_k \right)$$

- Binary classification: sigmoid
- Multi-class: softmax

$$f_m(x) = P(y = m|x) = \frac{e^{Z_m}}{\sum_{l=0}^M e^{Z_l}}$$

assuming M classes, an Z are the inputs received by the output layer

Turns raw numbers into probabilities

Training a NN

Conceptually simple: given a set of labelled data and a loss function, we need to optimise the weights and biases to minimise the loss.

- Forward propagation of errors.
 - Perturb weights and biases using gradient descent methods
 - Calculate final predictions and the loss
 - Perturb parameters again, until finding the optimal solution
 - Impossible for the number of parameters
- **Backpropagation**, or backward propagation of errors.
 - We compare the output with the actual labels and calculate the error.
 - Thanks to the chain rule, we can decompose the derivative and use gradient descent techniques to improve the weights and biases.

Training

- Let's consider our simple single layer NN and calculate the gradient of the loss as a function of the weights, ignoring the bias
- The input of a neuron can be written as $z_k = \sum_j w_{jk} A_j$
- The derivative of the loss L as a function of the weights is

$$\frac{dL}{dw_{jk}} = \frac{dL}{dA_k} \frac{dA_k}{dz_k} \frac{dz_k}{dw_{jk}}$$

- If a sigmoid is chosen as activation function

$$\frac{dA_k}{dz_k} = A_k(1 - A_k)$$

- If the L_2 is assumed for the loss function

$$L = \frac{1}{2} (y - y_{pred})^2 \rightarrow \frac{dL}{dA_k} = (A_k - y)$$

The derivative is $(A_k - y)A_k(1 - A_k) \frac{d \sum_j w_{jk} A_k}{dw_{jk}} = (A_k - y)A_k(1 - A_k)A_j$

- If it is not the output layer, but a hidden layer, you can get

$$\frac{dL}{dw_{ij}} = \left[\sum_k \delta_k w_{jk} \right] A_j (1 - A_j) A_i$$

Where $\delta_k = (A_k - y) A_k (1 - A_k)$

So, the gradient for each layer can be estimated from the gradient and the activation values of the next layer.

This is true for any differentiable activation function

Workflow

- Initialise randomly weights and biases
- Forward propagate the training data and calculate the first loss
- Estimate the derivatives of the network via backpropagation
- Update the weights as:

$$w'_{ij} = w_{ij} - \alpha \frac{dL}{dw_{ij}}$$

Where α is the learning rate and determines how much we perturb the weights and the biases.

Loss functions for classification

- Binary Cross-entropy

$$-\frac{1}{n} \sum_{i=1}^n [y_i \log(y_{i,pred}) + (1 - y_i) \log(1 - y_{i,pred})]$$

- Categorical Cross-entropy

$$-\sum_{i=1}^n \sum_{j=1}^k y_{ij} \log(y_{ij,pred})$$

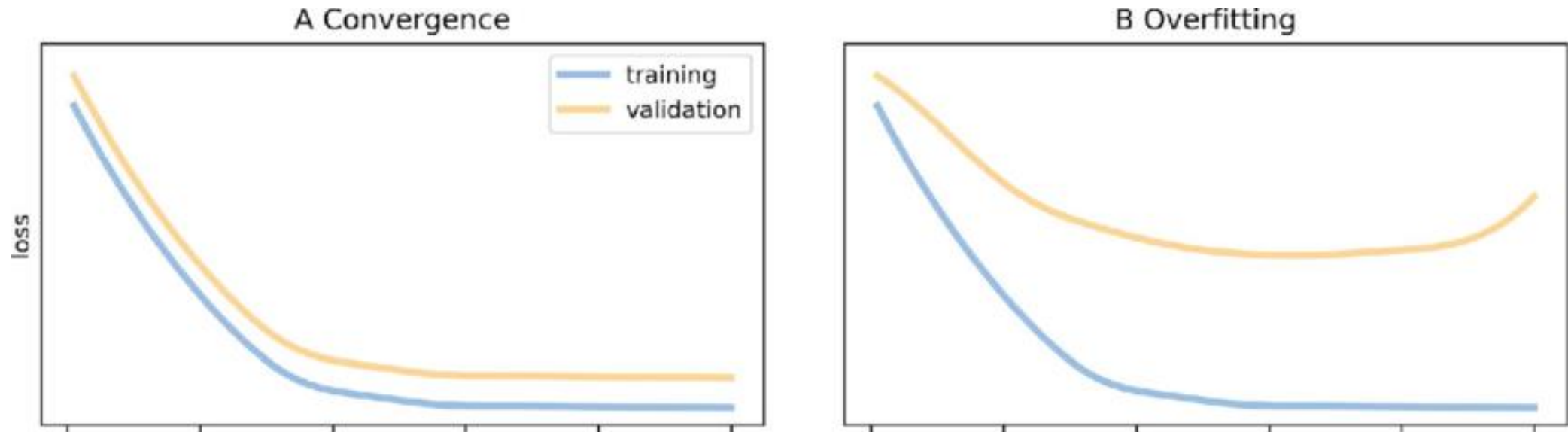
Training in practice

- **Number of epochs**: number of full passes through the NNs of the training data
- **Learning rate**: control the step size
- **Loss function**
- **Optimizers**: nowadays we use more than simple gradient descent (e.g. ADAM)
- Network **architecture**: number of layers, neurons per layers
- Train/validation/test splitting

Overfitting & Regularization in NNs

Even simple NNs have many parameters

- Very easy to overfit, especially with few labels
- Monitor loss validation curve



Overfitting & Regularization in NNs

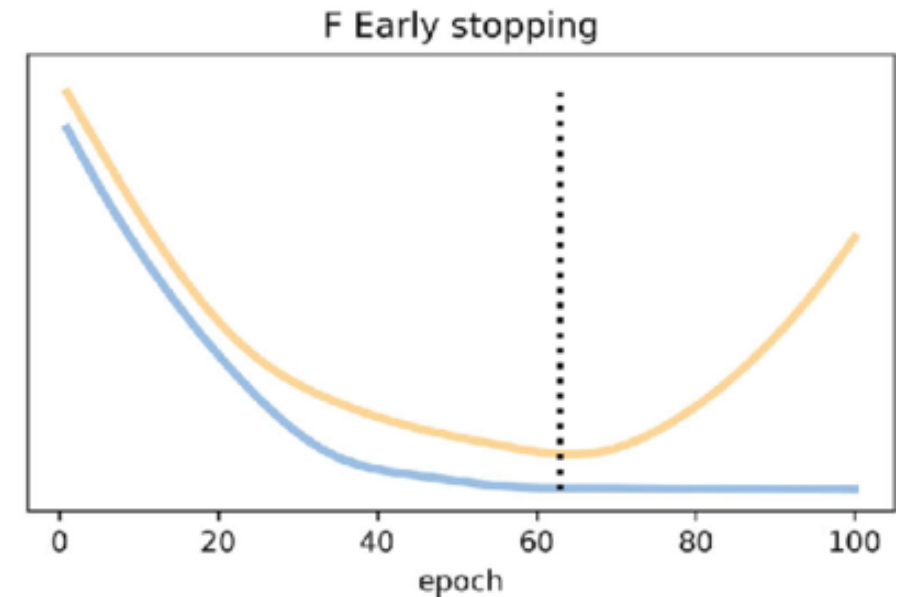
Dropout

- Randomly deactivate some neurons during training

Early stopping

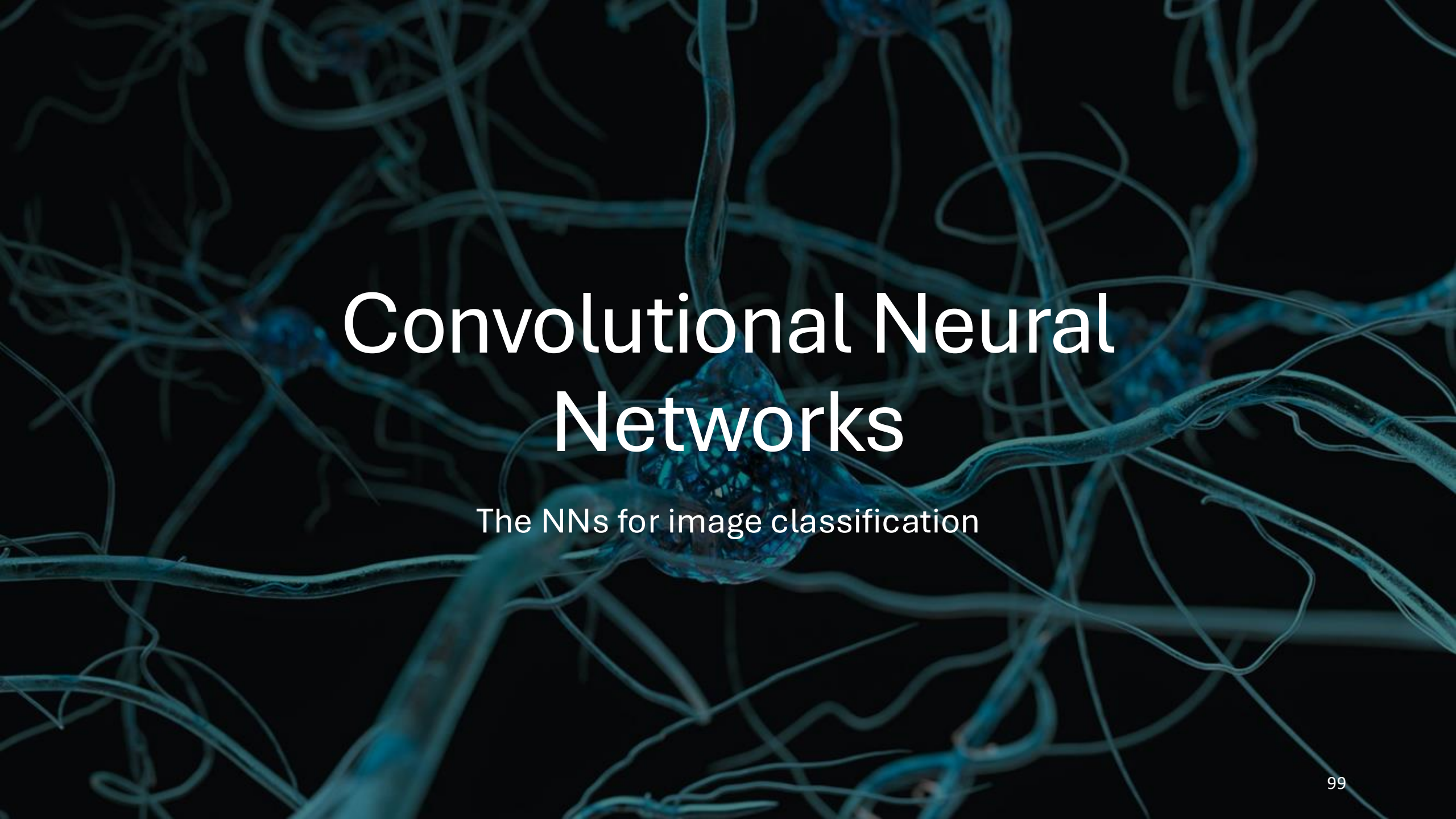
- Stop the training phase if there is no improvement on validation

Other more general regularization techniques



Final considerations for NNs

- Are data-hungry: require lot of data compared to classical ML
 - Often require data augmentation techniques to increase the training set
- Traditional NNs are not easy to apply to images
 - It would require the output of each pixel in an image to be connected to every neuron in the first layer of the network
 - If you think about the large astronomical images...this easily explodes

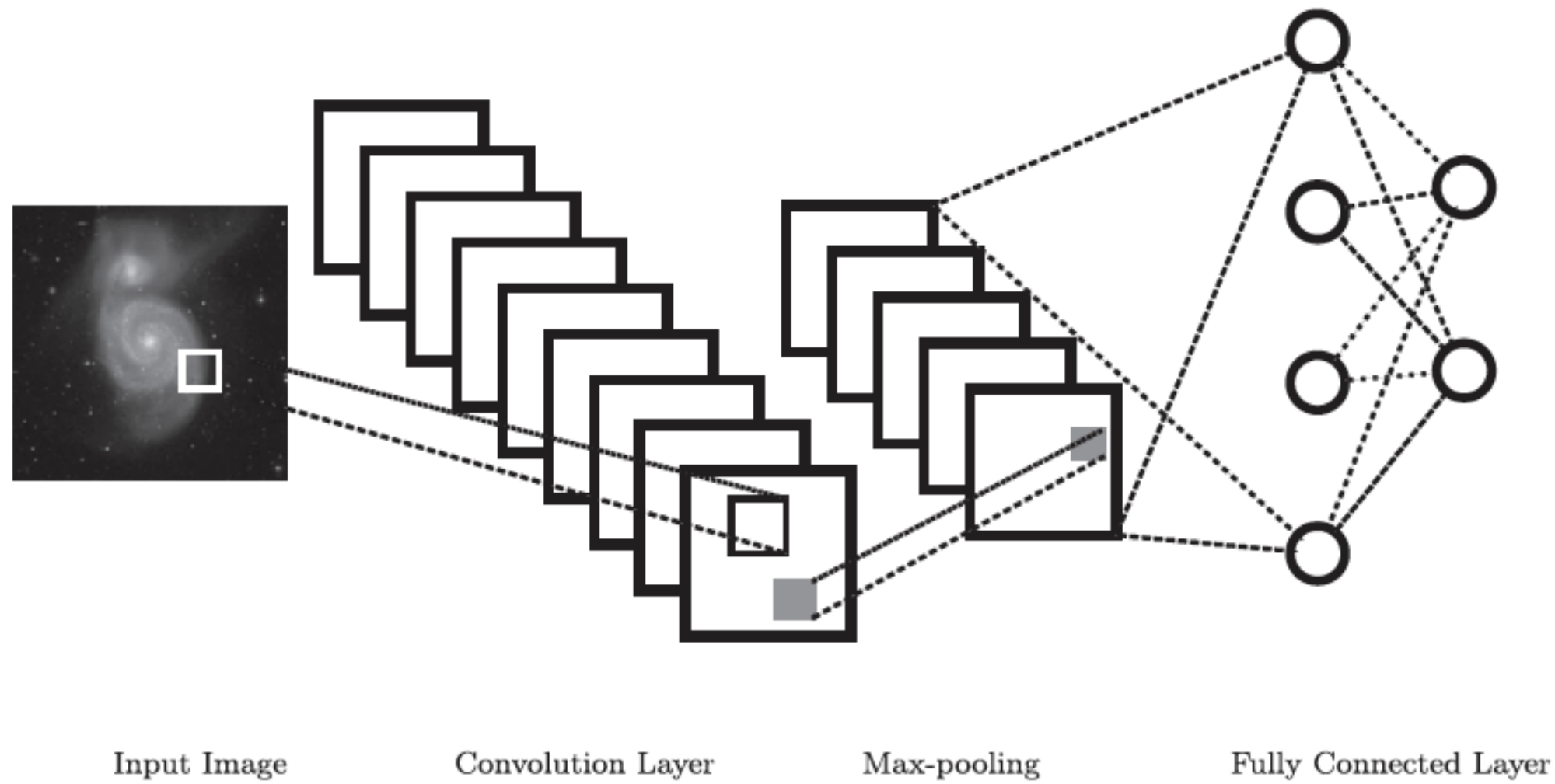


Convolutional Neural Networks

The NNs for image classification

CNN – intuition

- Reduces the complexity of the network by requiring that neurons only respond to inputs from a subset of an image
 - The receptive field
 - Mimics the operation of the visual cortex → focus on small regions
- Introduces **convolutional layers**
- **Pooling** or down-sampling operation
- A final fully connected layer for classification



Convolutional layer

- We convolve the input images, $I(x, y)$, with a **kernel** $K(x, y)$
- This will produce a **feature map** $F(x, y) = K(x, y) * I(x, y)$
- Kernels are described by their:
 - Depth: the number of kernels K
 - Size: its dimension
 - Stride: how many pixels a kernel shifts
 - Padding: number of zero-pixels added to each outer edge

Output of convolutional layer

- Image $N \times N$
- Kernel $m \times m$, stride s , padding p
- Output feature map:

$$\text{Output size} = \left\lceil \frac{N - m + 2 \times p}{s} \right\rceil + 1$$

- If we have multiple kernels, K , then we will have K feature maps

Example of convolution

1	1	1	0	0
0	1	1	1	0
0	0	1	1	1
0	0	1	1	0
0	1	1	0	0

1	0	1
0	1	0
1	0	1

1 _{x1}	1 _{x0}	1 _{x1}	0	0
0 _{x0}	1 _{x1}	1 _{x0}	1	0
0 _{x1}	0 _{x0}	1 _{x1}	1	1
0	0	1	1	0
0	1	1	0	0

Image

4		

Convolved
Feature

- We reduced the size of the input image (usually)
- We can stack multiple convolutional layers
- We then apply a non-linear activation function to each pixel of the resulting feature maps
 - Sigmoid
 - ReLU (very easy to differentiate)
- Training a CNN, means learn the underlying kernels (i.e. the values in the kernels), and the weights and biases in the fully-connected part

Pooling layer

- To further down-sample the feature maps (or cubes), pooling layers were introduced
- **Pooling summarises the feature map** values within a region of interest (i.e. a given window, like 2×2)
 - The summary is set by the user.
 - Average
 - Maximum
- You can set stride, padding, size

2	2	7	3
9	4	6	1
8	5	2	4
3	1	2	6

Max Pool

Filter - (2 x 2)

Stride - (2, 2)

9	7
8	6

2	2	7	3
9	4	6	1
8	5	2	4
3	1	2	6

Average Pool

Filter - (2 x 2)

Stride - (2, 2)

4.25	4.25
4.25	3.5

Fully-connected layers

- After convolution and pooling layers, we reduced our images to point where we can flatten them into a single array
- We use these flattened feature maps as input for fully-connected layers, where we make the classification
- Classification is performed as for NNs

Data Augmentation

- CNNs require a lot of data
- We can enlarge the image sample using label-preserving transformations
 - Random rotations
 - Flips
 - Small translations
 - Brightness/contrast enhancement

The transformation should reflect a symmetry variation the network should be invariant to

Transfer learning

Training a CNN from scratch requires large labeled datasets

- Often impractical when expert-labeled astronomical data

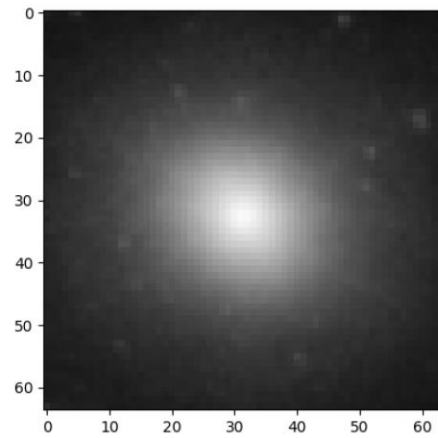
Core idea: **reuse a network already trained** on a large dataset (e.g., ImageNet) as a starting point, rather than initializing weights randomly.

➤ Domain Adaptation

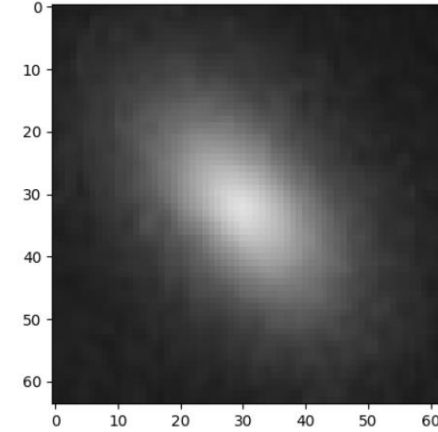
- **Feature extraction:** freeze the pretrained base, only train a new classifier head on top
 - Fast, works well with very little data
- **Fine-tuning:** unfreeze some (or all) of the pretrained layers and continue training at a low learning rate
 - More flexible, needs somewhat more data to avoid overfitting

CNNs for morphological classification

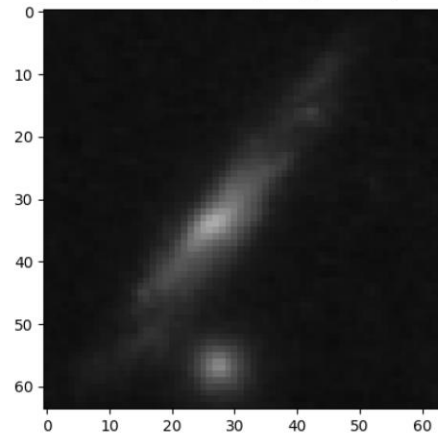
Predicted: 0 (Round smooth)



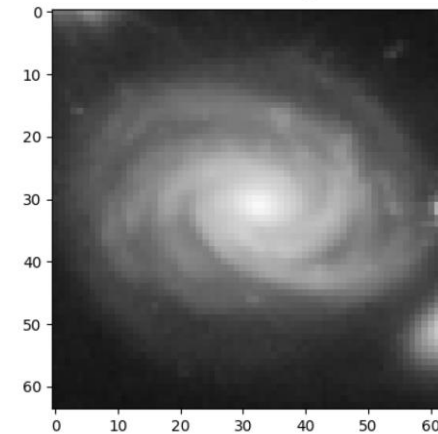
Predicted: 1 (In-between smooth)

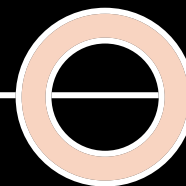
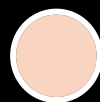


Predicted: 3 (Edge-on)



Predicted: 4 (Spiral)





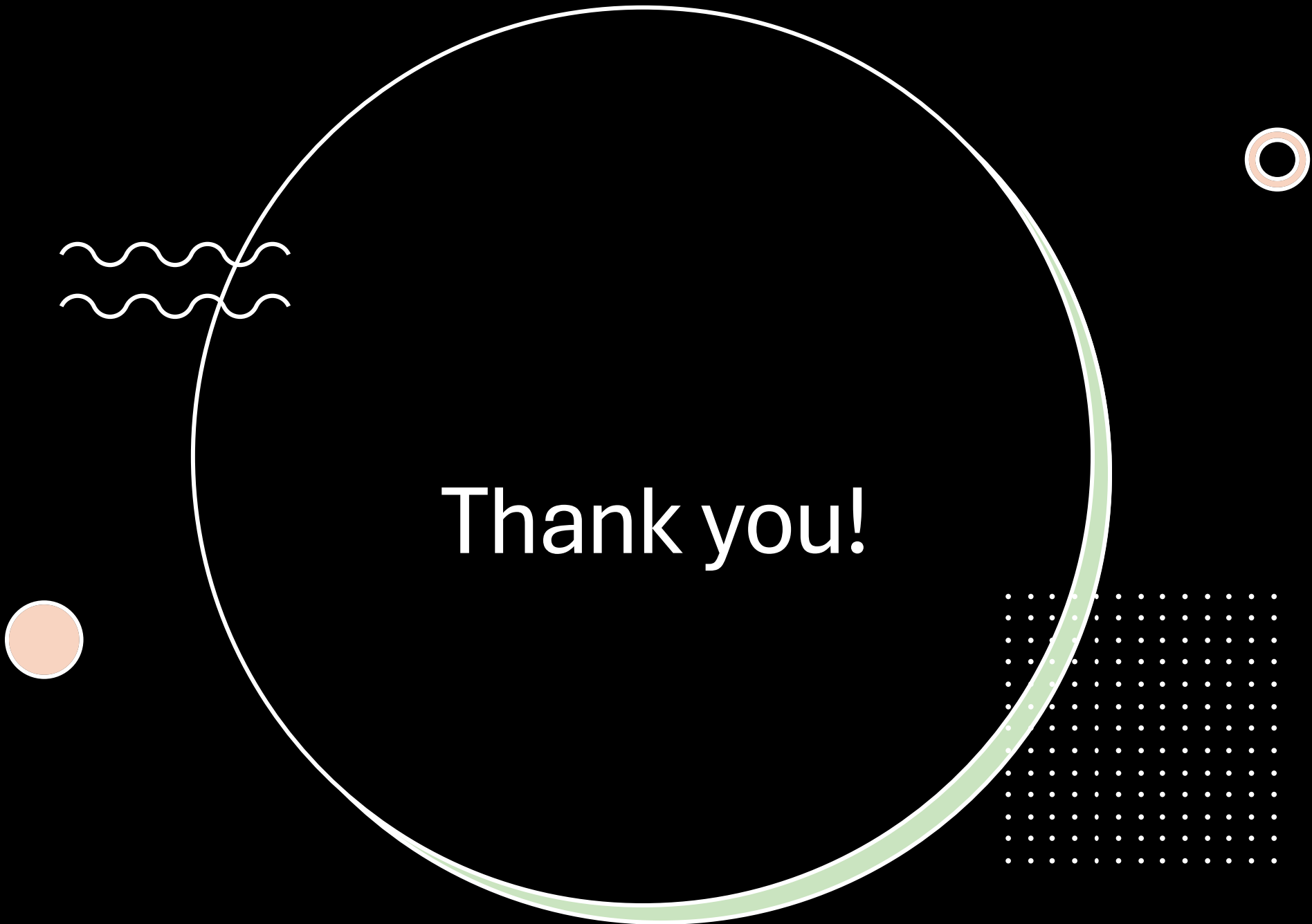
Final remarks

ML for classification



Choose the model that fits the question

- **Frame the task** — define the features, labels, scientific objective, and evaluation metric before training.
- **Start simple** — kNN offers an intuitive baseline; decision trees add interpretable, non-linear rules.
- **Control variance** — bagging and random forests average diverse trees; boosting corrects mistakes sequentially.
- **Match architecture to data** — neural networks capture complex patterns; CNNs exploit spatial structure in images.



Thank you!